Applied Bayesian Inference Project

A 10-Week Journey from Belief to Computation

By
Mohammad Mahdi Barzkar
Mohammad Mahdi Fotuhi
Mohammad Yasin Karimi
Hadi Zehtab

Course: Probability and Statistics

February 23, 2026

Contents

I Module 1: Foundations of Belief **4**

1 Week 1: The Mechanics of Uncertainty **4**
1.1 Theory and Intuition 4
1.1.1 Conditional Probability: The “Universe Shrinking” Tool 4
1.1.2 Bayes’ Theorem: The Learning Engine 4
1.2 Week 1 Deliverables 5

2 Week 2: Updating Beliefs – The Discrete Case **7**
2.1 Theory and Intuition 7
2.1.1 The “Cookie Problem”: A Bayesian Rosetta Stone 7
2.1.2 The Bayes Table Method 7
2.2 Week 2 Deliverables 8

II Module 2: Continuous Probability **15**

3 Week 3: The Beta Distribution – The Shape of Uncertainty **16**
3.1 Theory and Intuition 16
3.1.1 The Beta Distribution: The ”Hello World” of Bayesian ML 16
3.1.2 Conjugate Priors: The Algebraic Shortcut 16
3.2 Week 3 Deliverables 17

4 Week 4: Quantifying Risk – The Credible Interval **26**
4.1 Theory and Intuition 26
4.1.1 The ”Confidence” vs. ”Credible” Debate 26
4.1.2 ETI vs. HDI: How to cut the cake 27
4.2 Week 4 Deliverables 27

III Module 3: Bayesian Decision Making **34**

5 Week 5: Bayesian A/B Testing & Decision Theory **35**
5.1 Theory and Intuition . . . 35
5.1.1 The Bayesian Shift: Parameters are Random Variables . . . 35
5.1.2 The "Delta" Method: Subtracting Random Variables . . . 35
5.1.3 Decision Theory: Expected Loss . . . 35
5.2 Week 5 Deliverables . . . 36

6 Week 6: Model Checking – The "Turing Test" for Statistics **38**
6.1 Theory and Intuition . . . 38
6.1.1 Posterior Predictive Checks (PPC) . . . 38
6.1.2 The Euro Problem . . . 38
6.2 Week 6 Deliverables . . . 39

IV Module 4: Bayesian Classification **41**

7 Week 7: The Naive Bayes Classifier **41**
7.1 Theory and Intuition . . . 41
7.1.1 The "Bag of Words" Model . . . 41
7.1.2 The "Naive" Assumption . . . 42
7.1.3 Laplace Smoothing: The "Zero Probability" Problem . . . 42
7.2 Week 7 Deliverables . . . 42

8 Week 8: The Art of Evaluation **46**
8.1 Theory and Intuition . . . 46
8.1.1 The Confusion Matrix . . . 46
8.1.2 Precision vs. Recall: The Trade-off . . . 47
8.2 Week 8 Deliverables . . . 47

9 Week 9: Final Integration and Reflection **50**
9.1 Synthesis . . . 50
9.1.1 The Bayesian Advantage Summary . . . 50
9.2 Week 9 Deliverables . . . 50

Introduction

Welcome to the Applied Bayesian Inference Project. Over the next 10 weeks, you will transition from the rigid world of frequentist statistics—where parameters are fixed constants—to the flexible, dynamic world of Bayesian statistics—where parameters are uncertain variables described by probability distributions.

This is not just a theoretical exercise. You will build spam filters, analyze business decisions using A/B testing, and understand how machines learn from data.

The Core Philosophy

Imagine you hear a strange noise in your house.

- **Frequentist approach:** You calculate the frequency of strange noises in all houses over the last 100 years to determine if your house is haunted.
- **Bayesian approach:** You start with a "prior" belief (e.g., houses are rarely haunted). You gather data (the wind is blowing, the cat is missing). You update your belief (it's probably the wind, or maybe the cat).

Bayesian inference is the mathematical formalization of *learning*.

Part I
Module 1: Foundations of Belief

1 Week 1: The Mechanics of Uncertainty

1.1 Theory and Intuition

Before we can update beliefs, we must establish the fundamental rules of probability. We must understand how to measure the "space of possibilities."

1.1.1 Conditional Probability: The "Universe Shrinking" Tool

The notation $P(A|B)$ is read as "the probability of A, *given* that we accept B as truth."

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Deep Intuition: Imagine the probability space is a dartboard. $P(A)$ is the size of the target A. When we learn that event B has occurred, we throw away the rest of the dartboard. B becomes our *new universe*. We then ask: "Inside this new universe B, how much space does A occupy?"

1.1.2 Bayes' Theorem: The Learning Engine

Derived directly from conditional probability, this theorem is the engine of our entire course. It allows us to invert dependencies: we want to go from "The probability of the Data given the Hypothesis" (easy to calculate) to "The probability of the Hypothesis given the Data" (what we actually care about).

$$P(H|D) = \frac{P(D|H)P(H)}{P(D)}$$

Where:

- H: Hypothesis (e.g., "The coin is biased").
- D: Data (e.g., "I observed 3 heads in a row").
- $P(H)$: **Prior**. What did you believe yesterday?
- $P(D|H)$: **Likelihood**. If your theory were true, how surprised would you be by this data? (High likelihood = Low surprise).
- $P(D)$: **Evidence**. The total probability of seeing this data under *all possible explanations*.
- $P(H|D)$: **Posterior**. What you believe today.

Intuition Check: The Medical Test Paradox

Scenario: A rare disease affects 1% of the population ($P(D) = 0.01$). A test is 99% accurate (True Positive = 99%, False Positive = 1%). You test positive. **Intuitive Trap:** "I am 99% sure I have the disease." **Bayesian Reality:** You are only **50%** sure.
Why? Imagine 1,000 people.

- **The Sick:** 10 people have the disease. The test catches $\approx$ 10 of them (99% of 10).
- **The Healthy:** 990 people are healthy. The test wrongly flags $\approx$ 10 of them (1% of

990).

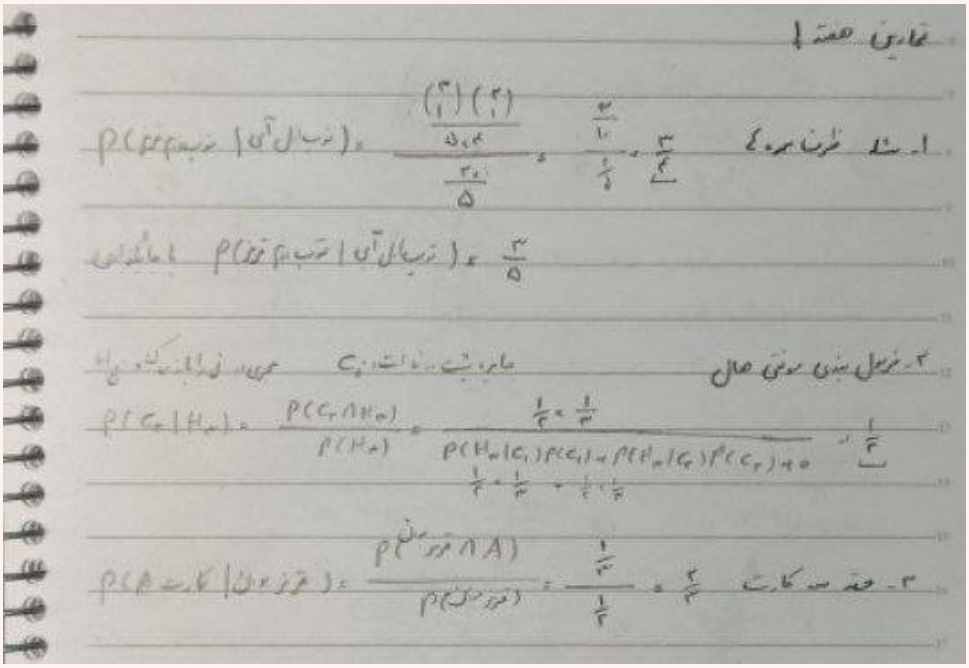
You tested positive. You are in the pool of 20 positive results. Only 10 of those 20 actually have the disease. Bayes' theorem forces us to respect the *base rate*.

1.2 Week 1 Deliverables

Week 1 Assignments

Part A: Theoretical Questions

1. **The Urn Draw.** A bag contains 3 red balls and 2 blue balls. You draw two balls sequentially *without* replacement.
   - Calculate the probability the second ball is red given the first was blue.
   - *Check:* Does the probability change if we draw *with* replacement? Why?
2. **The Monty Hall Formalization.** You select Door 1. The Host opens Door 3 to reveal a Goat. Define C_i as "Car behind Door i" and H_j as "Host opens Door j." Calculate $P(C_2|H_3)$ using Bayes' Theorem. Assume the host *must* open a door with a goat and cannot open your door.
3. **The Three-Card Swindle.** There are three cards in a hat: Card A (Red/Red), Card B (White/White), and Card C (Red/White). You pull a card blindly and place it on the table. The face showing is **Red**. What is the probability that the *other* side is also Red? *(Hint: It is not 1/2. Count the sides, not the cards.)*



Part B: Simulation Task (The Bayesian Coin) Write a Python script to simulate how a Bayesian agent "learns" truth.

- **Setup:** Create a "Secret Coin" with a true bias of $p = 0.7$.
- **Prior:** Assume a uniform prior (you believe all biases 0 to 1 are equally likely).

- **Process:** Flip the coin $N = 100$ times. After *every* flip, update the posterior distribution (using the Beta distribution).
- **Output:** Plot the probability density of your belief after 0, 5, and 100 flips. Show how the curve moves from a flat line to a sharp peak at 0.7.

  article listings xcolor geometry a4paper, margin=1in

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

true_p = 0.7
N = 100
np.random.seed(42)

alpha = 1
beta_param = 1

posterior_history = [(alpha, beta_param)]

for i in range(N):
    toss = np.random.rand() < true_p
    if toss:
        alpha += 1
    else:
        beta_param += 1
    posterior_history.append((alpha, beta_param))

p_vals = np.linspace(0, 1, 500)

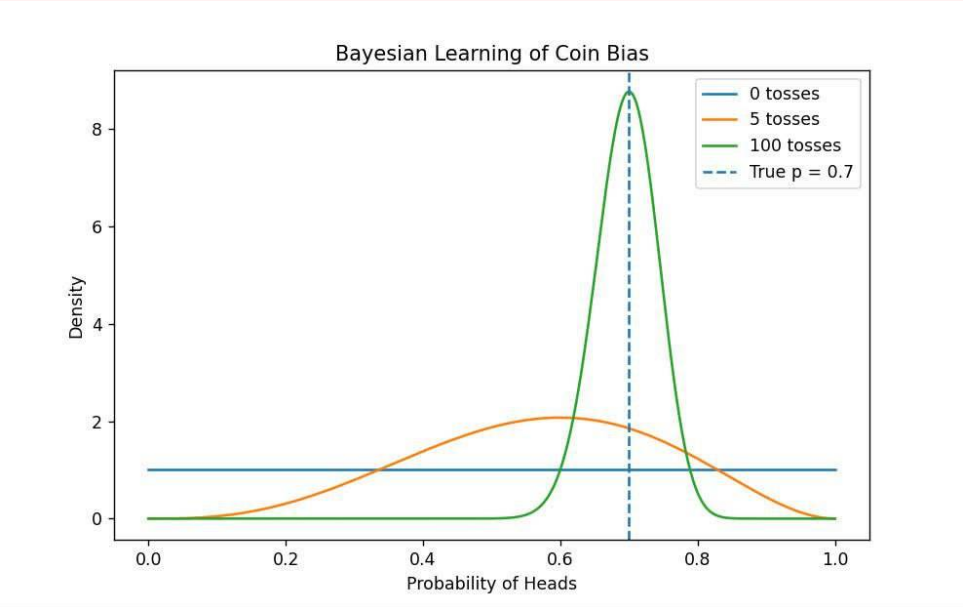
plt.figure(figsize=(8,5))

for k in [0, 5, 100]:
    a, b = posterior_history[k]
    plt.plot(p_vals, beta.pdf(p_vals, a, b), label=f"{k} tosses")

plt.axvline(true_p, linestyle='--', label="True p = 0.7")

plt.title("Bayesian Learning of Coin Bias")
plt.xlabel("Probability of Heads")
plt.ylabel("Density")
plt.legend()
plt.show()
```

Output:



2 Week 2: Updating Beliefs – The Discrete Case

2.1 Theory and Intuition

In Week 1, we established the philosophy. Now, we build the machinery. We apply Bayes' Theorem to **Discrete Hypotheses** (where there are a finite number of explanations).

2.1.1 The "Cookie Problem": A Bayesian Rosetta Stone

This classic example demonstrates how a single observation shifts probability mass between competing hypotheses.

The Setup:

- **Bowl 1:** 30 Vanilla, 10 Chocolate. (75% Vanilla)
- **Bowl 2:** 20 Vanilla, 20 Chocolate. (50% Vanilla)

You pick a bowl at random (Prior $P(B_1) = P(B_2) = 0.5$). You draw one cookie. It is **Vanilla**. Which bowl is it?

2.1.2 The Bayes Table Method

Algebra is prone to error. In computational statistics, we use the **Table Method**. Think of this as a competition where hypotheses "bet" on the data.

Hypothesis H	**Prior** $P(H)$	**Likelihood** $P(D\|H)$	**Unnorm. Posterior** $P(H) \times P(D\|H)$	**Posterior** $P(H\|D)$
Bowl 1	0.5	0.75	0.375	$0.375/0.625 = \mathbf{0.60}$
Bowl 2	0.5	0.50	0.250	$0.250/0.625 = \mathbf{0.40}$
Sum	**1.0**		**0.625** ($P(D)$)	**1.0**

Iterative Updating

Bayesian inference has memory. If we put the cookie back and draw again, we don't start with 50/50 beliefs. We start with our *updated* belief (60/40). This allows a Bayesian agent to accumulate knowledge over time.

2.2 Week 2 Deliverables

Week 2 Assignments

Part A: Theoretical Questions

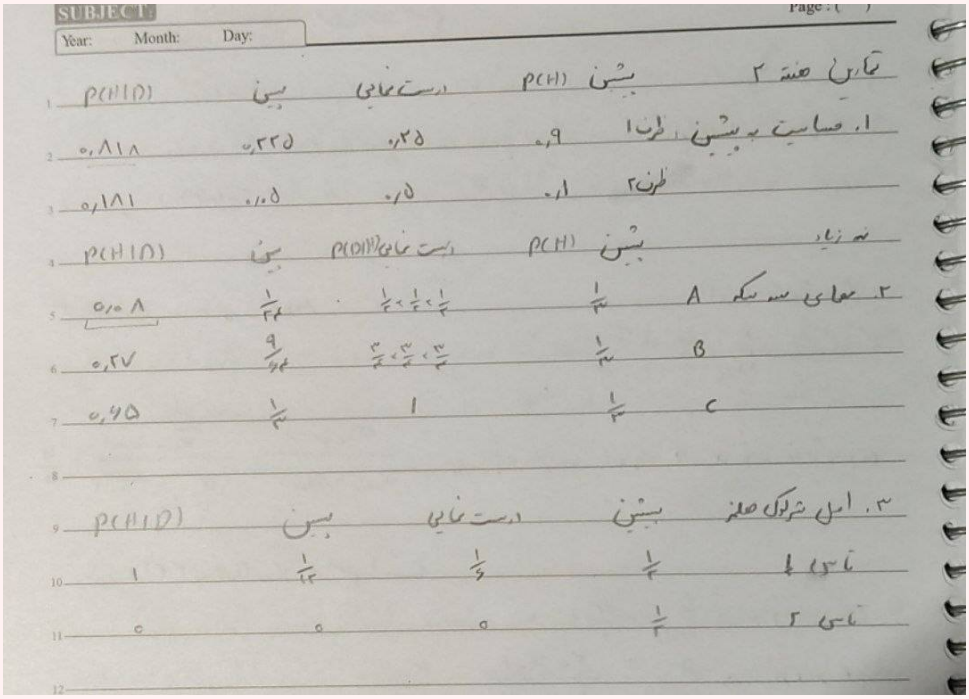
1. **(Easy) Sensitivity to Priors.** Revisit the Cookie Problem. Suppose you are nearly certain you picked Bowl 1, so your Prior for Bowl 1 is 0.9 (and Bowl 2 is 0.1). You draw a **Chocolate** cookie. Calculate the new posterior probability for Bowl 1. Does the data stick enough to shake your strong prior belief?

2. **(Medium) The Three-Coin Puzzle.** You have a box with three coins:

   - Coin A: Fair ($P(H) = 0.5$).
   - Coin B: Biased ($P(H) = 0.75$).
   - Coin C: Double-Headed ($P(H) = 1.0$).

   You pick one coin at random and flip it 3 times. The result is **Heads, Heads, Heads**. What is the probability you picked the Fair coin? (Hint: Calculate the likelihood of HHH for each coin first).

3. **(Hard) The Sherlock Holmes Principle.** You have a 4-sided die and a 6-sided die. You pick one at random. You roll it and get a **5**. Use a Bayes Table to prove that the probability of the die being 4-sided becomes exactly 0. *Note: This demonstrates Falsification. If the Likelihood is 0, the Posterior is 0, regardless of the Prior.*



Part B: Simulation Task (Introduction to Machine Learning)
Task: Bayesian Linear Regression (Grid Approximation)
In Machine Learning, "Regression" is usually taught as finding the single best line that fits the data (minimizing error). In Bayesian ML, we don't find one line; we find the *probability distribution over all possible lines*.
We assume a data model: $y = mx + c +$ noise. Our goal is to learn the joint probability distribution of the Slope (m) and Intercept (c).

1. **Generate Synthetic Data:** Create a "Ground Truth" function: $y = 1.5x + 2.0$. Generate 10 random (x, y) points with added Gaussian noise ($\sigma = 1.0$).

```
import matplotlib.pyplot as plt

np.random.seed(42)
m_true, c_true, sigma = 1.5, 2.0, 1.0

x = np.random.uniform(-2, 3, 10)
y_true = m_true * x + c_true
noise = np.random.normal(0, sigma, 10)
y = y_true + noise

print("Ground Truth: y = 1.5x + 2.0, σ = 1.0\n")
print("Index     x          y          y_true     Noise")
print("-" * 55)
for i, (xi, yi, yti, ni) in enumerate(zip(x, y, y_true, noise), 1):
    print(f"{i:2d}     {xi:8.4f}   {yi:8.4f}   {yti:8.4f}   {ni:8.4f}"
    ↪)

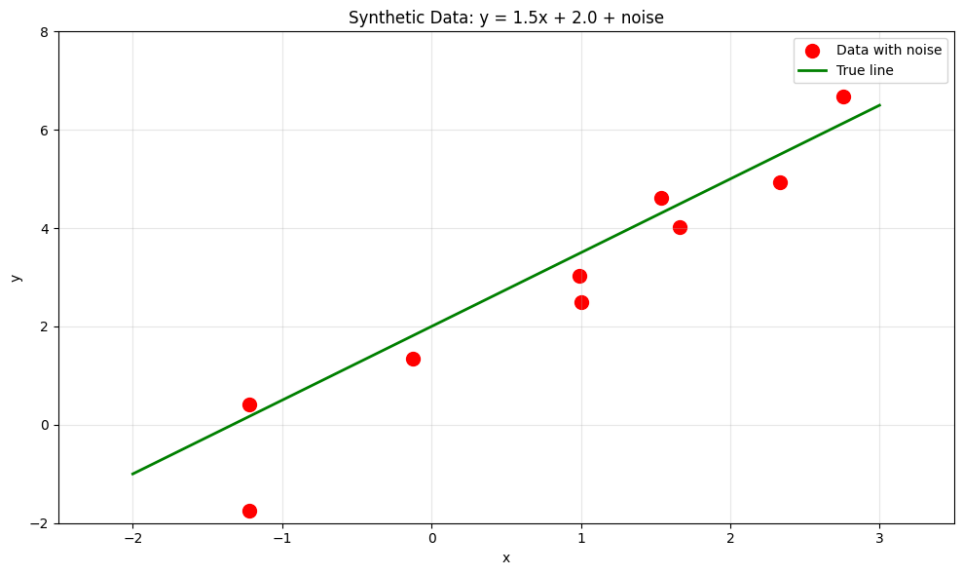
print(f"\nNoise mean: {np.mean(noise):.4f}, Noise std: {np.std(noise)
    ↪:.4f}")
print(f"Correlation: {np.corrcoef(x, y)[0,1]:.4f}")

plt.figure(figsize=(10,6))
plt.scatter(x, y, color='red', s=100, label='Data with noise')
plt.plot(np.linspace(-2,3,100), m_true*np.linspace(-2,3,100)+c_true, '
    ↪g-', linewidth=2, label='True line')
plt.xlabel('x'); plt.ylabel('y'); plt.title('Synthetic Data: y = 1.5x
    ↪+ 2.0 + noise')
plt.legend(); plt.grid(alpha=0.3); plt.xlim(-2.5,3.5); plt.ylim(-2,8)
plt.tight_layout()
plt.show()
```

Output:
Ground Truth: y = 1.5x + 2.0, σ = 1.0

Index	x	y	y_true	Noise
1	-0.1273	1.3396	1.8091	-0.4695
2	2.7536	6.6729	6.1304	0.5426
3	1.6600	4.0265	4.4900	-0.4634
4	0.9933	3.0242	3.4899	-0.4657
5	-1.2199	0.4121	0.1701	0.2420
6	-1.2200	-1.7433	0.1700	-1.9133
7	-1.7096	-2.2893	-0.5644	-1.7249
8	2.3309	4.9340	5.4963	-0.5623
9	1.0056	2.4955	3.5084	-1.0128
10	1.5404	4.6248	4.3105	0.3142

Noise mean: -0.5513, Noise std: 0.7773
Correlation: 0.9737



2. **Define the Prior (The Grid):** Create a 50×50 matrix representing the grid of joint probabilities.

   - Range for m: $[0, 3]$
   - Range for c: $[-2, 5]$

   Initialize with a Uniform Prior (all cells $= 1/2500$).

3. **The Likelihood Function (The ML Core):** In Least Squares regression, we minimize distance. In Bayesian regression, we maximize the Gaussian Likelihood. For a single data point (x_{obs}, y_{obs}) and a specific hypothesis (m, c), the likelihood is:

$$P(y_{obs}|x_{obs}, m, c) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y_{obs}-(mx_{obs}+c))^2}{2\sigma^2}}$$

   (Hint: Use `scipy.stats.norm.pdf`*. The "error" is the distance between the observed point and the line proposed by the hypothesis.)*

```python
import numpy as np
from scipy.stats import norm

print("="*60)
print("LIKELIHOOD FUNCTION (sigma = 1.0)")
print("="*60)

m_true, c_true, sigma = 1.5, 2.0, 1.0  # □□ sigma □□□□□ 1.0
x_obs, y_obs = -0.1273, 1.5743

def likelihood(x, y, m, c, sigma):
    return norm.pdf(y, loc=m*x + c, scale=sigma)

hypotheses = [(1.5, 2.0, "Good"), (1.4, 2.1, "Decent"), (0.5, 3.0, "
    ↪Bad")]

print(f"\nPoint: x={x_obs:.4f}, y={y_obs:.4f}")
print(f"sigma = {sigma}")
print("-"*60)
for m, c, label in hypotheses:
    like = likelihood(x_obs, y_obs, m, c, sigma)
    error = y_obs - (m*x_obs + c)
    print(f"{label}: m={m}, c={c}, error={error:.4f}, likelihood={like
    ↪:.6f}")

m_range = np.linspace(0, 3, 50)
c_range = np.linspace(-2, 5, 50)
M, C = np.meshgrid(m_range, c_range)

like_grid = likelihood(x_obs, y_obs, M, C, sigma)
max_idx = np.unravel_index(like_grid.argmax(), like_grid.shape)

print("\n" + "-"*60)
print(f"Max likelihood: m={M[max_idx]:.2f}, c={C[max_idx]:.2f}")
print(f"True values:    m={m_true:.2f}, c={c_true:.2f}")
print(f"sigma used: {sigma}")

```

Output:

$$
\begin{aligned}
\text{Point:} &\quad (x, y) = (-0.1273, 1.5743), \quad \sigma = 1.0 \\
\text{Good:} &\quad m = 1.5,\ c = 2.0, \quad \text{error} = -0.2348, \quad \mathcal{L} = 0.388100 \\
\text{Decent:} &\quad m = 1.4,\ c = 2.1, \quad \text{error} = -0.3475, \quad \mathcal{L} = 0.375570 \\
\text{Bad:} &\quad m = 0.5,\ c = 3.0, \quad \text{error} = -1.3620, \quad \mathcal{L} = 0.157784 \\
\text{Max likelihood:} &\quad \hat{m}_{\text{ML}} = 1.10,\ \hat{c}_{\text{ML}} = 1.71 \\
\text{True values:} &\quad m_{\text{true}} = 1.50,\ c_{\text{true}} = 2.00
\end{aligned}
$$

4. **The Update Loop:**
   - Iterate through your 10 data points one by one.
   - For each data point, calculate the Likelihood for **every cell** in your 50×50 grid.
   - Multiply: $\text{Posterior}_{grid} = \text{Prior}_{grid} \times \text{Likelihood}_{grid}$.

- Normalize the grid so it sums to 1.

```python
import numpy as np
from scipy.stats import norm

np.random.seed(0)
x = np.linspace(-2, 3, 10)
y = 1.5*x + 2.0 + np.random.normal(0, 0.5, len(x))
sigma = 0.5

m_vals = np.linspace(0, 3, 50)
c_vals = np.linspace(-2, 5, 50)
M, C = np.meshgrid(m_vals, c_vals)

posterior = np.ones_like(M)
posterior /= posterior.sum()

print("Step |   m_MAP   c_MAP   |  MaxProb   Entropy")
print("-"*55)

for step, (xi, yi) in enumerate(zip(x, y), 1):
    likelihood = norm.pdf(yi, M*xi + C, sigma)
    posterior *= likelihood
    posterior /= posterior.sum()

    idx = np.unravel_index(np.argmax(posterior), posterior.shape)
    m_map, c_map = M[idx], C[idx]
    entropy = -np.sum(posterior * np.log(posterior + 1e-12))

    print(f"{step:>4} | {m_map:7.3f}  {c_map:7.3f} | "
          f"{posterior.max():8.6f}  {entropy:8.3f}")

m_mean = np.sum(M * posterior)
c_mean = np.sum(C * posterior)

print("\nFinal Estimates:")
print(f"MAP estimate     : m = {m_map:.3f}, c = {c_map:.3f}")
print(f"Posterior mean   : m = {m_mean:.3f}, c = {c_mean:.3f}")
print("True parameters  : m = 1.500, c = 2.000")

#additional data
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 5))
plt.subplot(1,2,1)
plt.imshow(posterior, extent=[0,3,-2,5], origin='lower', cmap='hot')
plt.plot(1.5, 2.0, 'b+', markersize=15)
plt.colorbar()
plt.title('Final Posterior')

plt.subplot(1,2,2)
plt.scatter(x, y, c='red')
x_line = np.linspace(-2, 3)
plt.plot(x_line, m_map*x_line + c_map, 'b-', label='MAP')
plt.plot(x_line, 1.5*x_line + 2.0, 'g--', label='True')
plt.legend()
plt.title('Data & MAP Line')
plt.tight_layout()
```

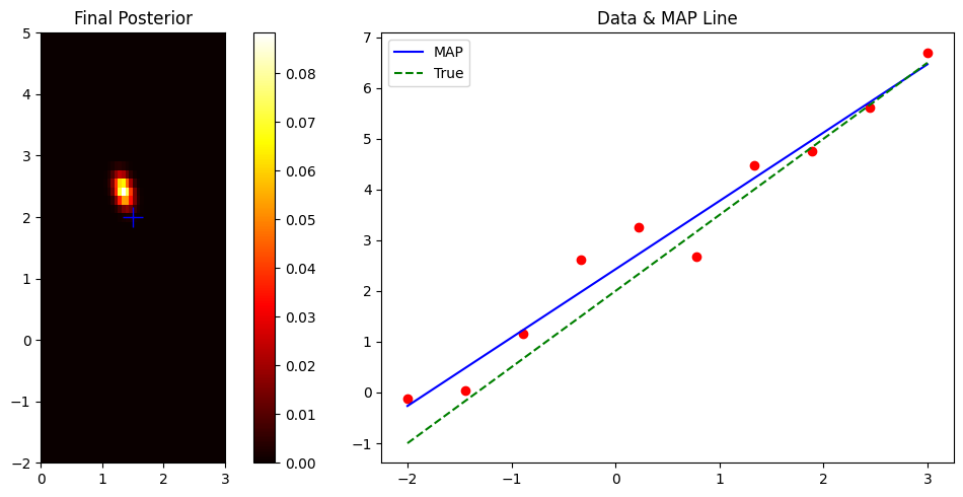
```
plt.show()

```

Output:

```
Step |   m_MAP    c_MAP    |  MaxProb    Entropy
-----------------------------------------------------
   1 |   0.061    0.000 | 0.002662     6.424
   2 |   0.184    0.286 | 0.005368     6.056
   3 |   1.163    2.000 | 0.007774     5.771
   4 |   1.653    2.857 | 0.013803     5.278
   5 |   1.653    2.857 | 0.021739     4.822
   6 |   1.286    2.429 | 0.031171     4.452
   7 |   1.347    2.429 | 0.042354     4.139
   8 |   1.347    2.429 | 0.055539     3.870
   9 |   1.286    2.429 | 0.069734     3.633
  10 |   1.347    2.429 | 0.088341     3.421

Final Estimates:
MAP estimate     : m = 1.347, c = 2.429
Posterior mean   : m = 1.347, c = 2.446
True parameters  : m = 1.500, c = 2.000
```



5. **Visualization (The "Learning" Moment):** Produce a plot with two side-by-side subplots:
   - **Left (Parameter Space):** A Heatmap of the 50×50 grid. Initially, it is one color (flat). By the end, it should be a bright "hot spot" centered at $m = 1.5, c = 2.0$.
   - **Right (Data Space):** Plot the 10 data points. Sample 50 random pairs of (m, c) from your posterior grid and draw those lines.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter

print("="*60)
```

```
print("QUESTION 5: FINAL VISUALIZATION")
print("="*60)

fig = plt.figure(figsize=(16, 7))

ax1 = fig.add_subplot(121)
im = ax1.imshow(posterior, extent=[0, 3, -2, 5], origin='lower',
                aspect='auto', cmap='hot', interpolation='bilinear')
ax1.set_xlabel('Slope (m)')
ax1.set_ylabel('Intercept (c)')
ax1.set_title('Left Panel: Final Posterior Distribution')
plt.colorbar(im, ax=ax1, label='Probability')

max_idx = np.unravel_index(posterior.argmax(), posterior.shape)
m_map = M[max_idx]
c_map = C[max_idx]

ax1.plot(m_true, c_true, 'b+', markersize=20, markeredgewidth=3, label
    ↪='True parameters')
ax1.plot(m_map, c_map, 'r*', markersize=15, label='MAP estimate')
ax1.legend()

smoothed = gaussian_filter(posterior, sigma=1.0)
threshold = np.percentile(smoothed, 68)
ax1.contour(M, C, smoothed, levels=[threshold], colors='white',
    ↪linewidths=2, linestyles='--')

# Right panel: Data space with sampled lines
ax2 = fig.add_subplot(122)

n_samples = 50
m_samples = []
c_samples = []

posterior_flat = posterior.flatten()
posterior_flat = posterior_flat / posterior_flat.sum()
indices = np.random.choice(len(posterior_flat), size=n_samples, p=
    ↪posterior_flat)

for idx in indices:
    i, j = np.unravel_index(idx, M.shape)
    m_samples.append(M[i, j])
    c_samples.append(C[i, j])

ax2.scatter(x_data, y_data, color='red', s=120, label='Observed data',
           edgecolors='black', linewidth=2, zorder=5)

x_line = np.linspace(-2, 3, 100)
for m, c in zip(m_samples, c_samples):
    y_line = m * x_line + c
    ax2.plot(x_line, y_line, 'b-', alpha=0.1, linewidth=1)

y_true_line = m_true * x_line + c_true
ax2.plot(x_line, y_true_line, 'g--', linewidth=3, label='True line',
    ↪color='green')

```

```
y_samples = np.array([m * x_line + c for m, c in zip(m_samples, c_samples)])
y_upper = np.percentile(y_samples, 97.5, axis=0)
y_lower = np.percentile(y_samples, 2.5, axis=0)
ax2.fill_between(x_line, y_lower, y_upper, alpha=0.2, color='blue', label='95% CI')

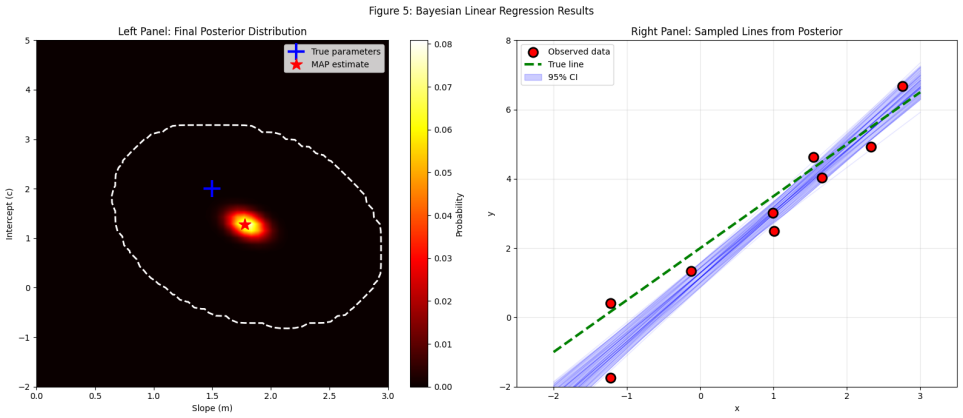
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Right Panel: Sampled Lines from Posterior')
ax2.legend(loc='upper left')
ax2.grid(True, alpha=0.3)
ax2.set_xlim(-2.5, 3.5)
ax2.set_ylim(-2, 8)

plt.suptitle('Figure 5: Bayesian Linear Regression Results')
plt.tight_layout()
plt.show()

print(f"\nFinal MAP estimate: m = {m_map:.2f}, c = {c_map:.2f}")
print(f"True parameters: m = {m_true:.2f}, c = {c_true:.2f}")
print(f"Absolute error: Δm = {abs(m_map - m_true):.3f}, Δc = {abs(c_map - c_true):.3f}")

```

Output:



Final MAP estimate: m = 1.78, c = 1.29
True parameters: m = 1.50, c = 2.00
Absolute error: Δm = 0.276, Δc = 0.714

Part II
Module 2: Continuous Probability

3 Week 3: The Beta Distribution – The Shape of Uncertainty

3.1 Theory and Intuition

In Module 1, we tested discrete hypotheses (Bowl 1 vs. Bowl 2). But in the real world, parameters are continuous. A website's conversion rate isn't just "High" or "Low"; it is a floating-point number anywhere between 0.0 and 1.0.

To do Bayesian inference on percentages, we need a probability distribution *of probabilities*. Enter the Beta Distribution.

3.1.1 The Beta Distribution: The "Hello World" of Bayesian ML

The Beta distribution is defined on the interval $[0, 1]$. It is parameterized by two numbers, α and β, which we call **Hyperparameters**.

$$f(p; \alpha, \beta) \propto p^{\alpha-1}(1-p)^{\beta-1}$$

The Intuition of Parameters: Instead of abstract Greek letters, think of α and β as *pseudo-counts*:

- $\alpha - 1 \approx$ Number of Successes (Heads/Clicks) observed.
- $\beta - 1 \approx$ Number of Failures (Tails/No-Clicks) observed.
- **Beta(1, 1):** Uniform (Flat). Represents "I have seen 0 successes and 0 failures." Total ignorance.
- **Beta(100, 100):** A Gaussian-like bell curve centered tightly at 0.5. Represents "I have seen a lot of data, and the coin is fair."
- **Beta(0.5, 0.5):** A U-shape. Represents "I suspect the coin is rigged to be either almost all Heads or almost all Tails, but I don't know which yet."

3.1.2 Conjugate Priors: The Algebraic Shortcut

In Machine Learning, we love the Beta distribution because it is the **Conjugate Prior** to the Binomial likelihood. **What this means:** If you start with a Beta prior and collect Binomial data (wins/losses), your Posterior is *guaranteed* to also be a Beta distribution.

$$\text{Posterior Beta}(\alpha', \beta') = \text{Beta}(\alpha_{prior} + \text{Successes}, \quad \beta_{prior} + \text{Failures})$$

Significance: This turns complex calculus (integration) into simple addition. This allows real-time updating in algorithms (like ad servers) doing millions of calculations per second.

ML Concept: "Smoothing" is just a Prior

Have you heard of **Laplace Smoothing** in Naive Bayes Classifiers? When calculating the probability of a word appearing in an email, we often add $+1$ to the count so we never

get a zero probability.

$$P(w) = \frac{\text{Count}(w) + 1}{N + 2}$$

Bayesian Reveal: This is exactly the expected value of a posterior using a **Beta(1,1)** prior! Adding $+1$ to the numerator and $+2$ to the denominator isn't a "hack"; it is a rigorous Bayesian update starting from a Uniform Belief.

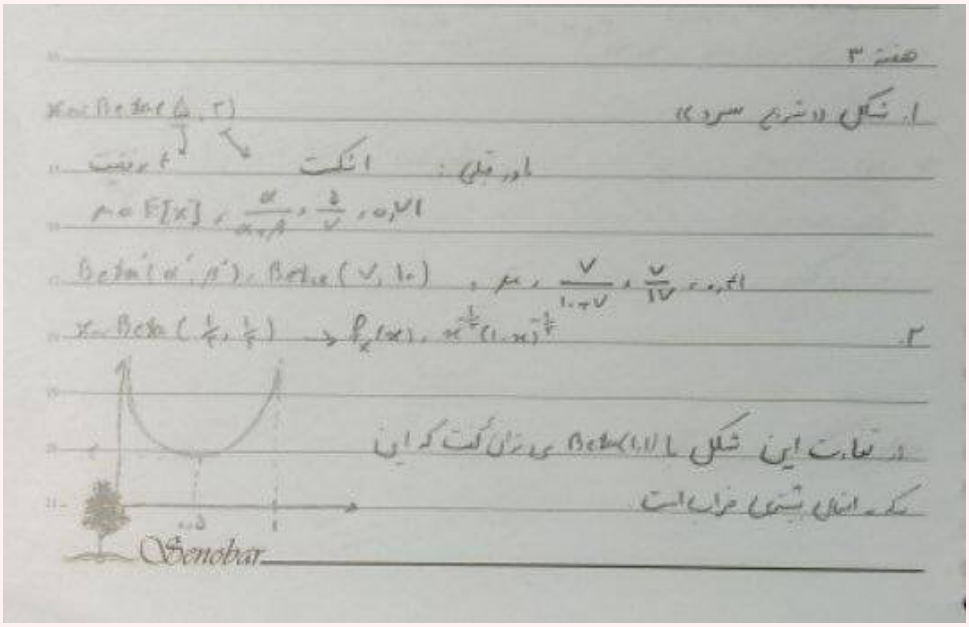
3.2 Week 3 Deliverables

Week 3 Assignments

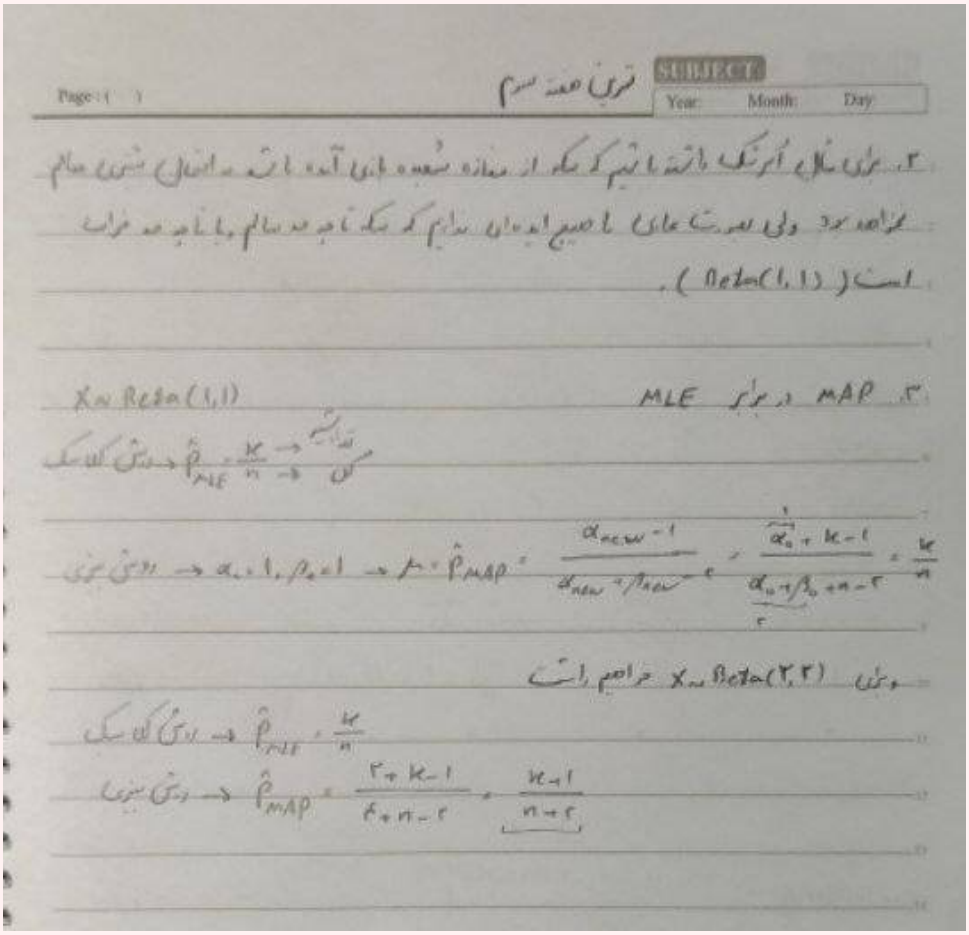
Part A: Theoretical Questions

1. **The "Cold Start" Problem.** You are launching a new product. You have no data, but your boss is optimistic.
   - You choose a Prior of Beta(5, 2). What does this imply about your pre-data belief (in terms of successes/failures)?
   - You survey 10 customers. 2 buy, 8 don't. Calculate the Posterior (α, β) and the new Expected Value (Mean).

2. **The Jeffreys Prior.** Plot a Beta(0.5, 0.5) distribution. Explain why this "U-shape" might be a more "objective" prior for a coin flip than Beta(1, 1) if you suspect the coin comes from a magician's shop. *Hint: It puts more weight on extreme biases.*



3. **MAP vs. MLE.** In Frequentist ML, we use Maximum Likelihood Estimation (MLE): $\hat{p} = \frac{k}{n}$. In Bayesian ML, we often use Maximum A Posteriori (MAP), which is the *mode* (peak) of the posterior.
   - Prove that if the Prior is Beta(1,1), the MAP estimate is identical to the MLE.
   - Prove that if the Prior is Beta(2,2), the MAP estimate pulls the result toward 0.5. (This is mathematically equivalent to **L2 Regularization**).



Part B: Simulation Task (Ranking & Recommendation Systems) *How do Reddit, Amazon, and Yelp rank items? They don't just use the average.*
The Scenario: You are the Chief Data Scientist for a movie streaming site. You need to build an algorithm to display the "Top Rated Movies" on the homepage. You have the following data (Likes vs. Dislikes) for 3 movies:

- **Movie A (The Unknown):** 2 Likes, 0 Dislikes. (100% Positive)
- **Movie B (The Classic):** 90 Likes, 10 Dislikes. (90% Positive)
- **Movie C (The Divisive):** 20 Likes, 20 Dislikes. (50% Positive)

Task: The "Lower Bound" Ranking Algorithm

1. **The Frequentist Failure:** Calculate the raw average rating ($\frac{\text{Likes}}{\text{Total}}$) for each movie. *Observation:* Notice that Movie A (100%) is ranked higher than Movie B (90%). Explain in one sentence why this is a bad user experience.

   Calculations:

$$\text{Movie A} : \frac{100}{100} = 1.00 = 100\%$$
$$\text{Movie B} : \frac{90}{100} = 0.90 = 90\%$$
$$\text{Movie C} : \frac{20}{40} \approx 0.50 = 50\%$$

Observation: Movie A (100%) is ranked higher than Movie B (90%).

Why is this a bad user experience?

> Movie A has only 100 votes and zero negative feedback, so its 100% rating is unreliable and could drop dramatically with just a few dislikes.

2. **The Bayesian Posterior:** Assume a neutral Prior of **Beta(2, 2)** (which is equivalent to adding +1 fake Like and +1 fake Dislike to smooth the data). Calculate the Posterior Beta parameters (α, β) for all three movies.

```python
from scipy.stats import beta
import numpy as np
import matplotlib.pyplot as plt

movies = {
    'A (Unknown)': (2, 0),
    'B (Classic)': (90, 10),
    'C (Divisive)': (20, 20)
}

print("=" * 70)
print("POSTERIOR PARAMETERS WITH BETA(2,2) PRIOR")
print("=" * 70)
print("\nPrior: Beta(2, 2) → +2 likes, +2 dislikes (smoothing)")
print("\nMovie Data with Posterior Parameters:")
print("-" * 70)
print(f"{'Movie':<15} {'Likes':<8} {'Dislikes':<10} α{' (posterior)':<15} β{' (posterior)':<15}")
print("-" * 70)

for name, (likes, dislikes) in movies.items():
    a = likes + 2
    b = dislikes + 2
    print(f"{name:<15} {likes:<8} {dislikes:<10} {a:<15} {b:<15}")

print("\n" + "=" * 70)
print("RANKING BY 5TH PERCENTILE (LOWER BOUND)")
print("=" * 70)

plt.figure(figsize=(10, 5))
x = np.linspace(0, 1, 1000)
colors = ['red', 'green', 'blue']
results = []

for i, (name, (likes, dislikes)) in enumerate(movies.items()):
    a = likes + 2
    b = dislikes + 2
    mean = a / (a + b)
    p5 = beta.ppf(0.05, a, b)
    results.append((name, p5))
    print(f"{name}: α={a}, β={b}, Mean={mean:.3f}, 5th%={p5:.3f}")

    plt.plot(x, beta.pdf(x, a, b), colors[i], linewidth=2, label=name)
    plt.axvline(p5, color=colors[i], linestyle='--', alpha=0.7)
```

```
    plt.fill_between(x[x <= p5], beta.pdf(x[x <= p5], a, b), alpha
    ↪=0.2, color=colors[i])

plt.xlabel('Rating')
plt.ylabel('Probability Density')
plt.title('Posterior Distributions with 5th Percentile (shaded area)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()

print("\n" + "-" * 50)
print("Final Ranking (by 5th Percentile):")
results.sort(key=lambda x: x[1], reverse=True)
for i, (name, p5) in enumerate(results, 1):
    print(f"{i}. {name}: {p5*100:.2f}%")
```

Output:

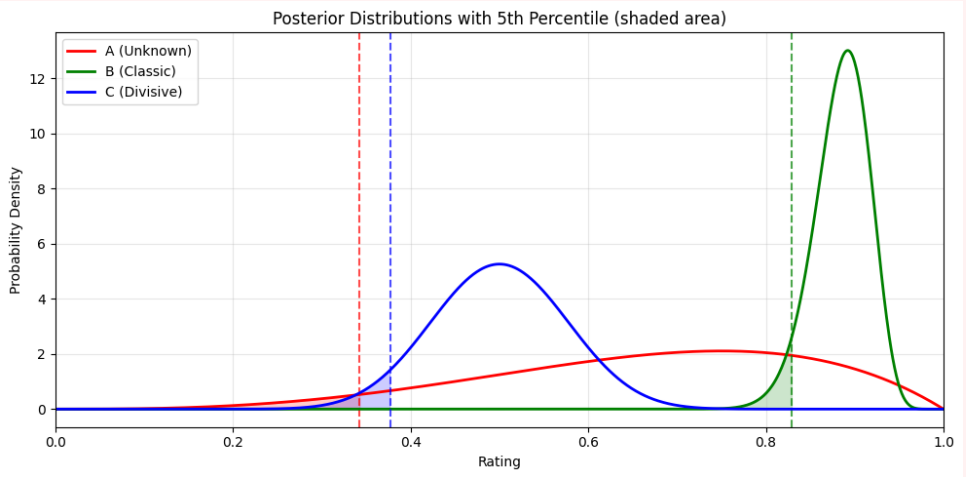
POSTERIOR PARAMETERS WITH BETA(2,2) PRIOR

Prior: Beta(2, 2) $\rightarrow$ +2 likes, +2 dislikes (smoothing)

Movie	Likes	Dislikes	α (posterior)	β (posterior)
A (Unknown)	2	0	4	2
B (Classic)	90	10	92	12
C (Divisive)	20	20	22	22

RANKING BY 5TH PERCENTILE (LOWER BOUND)

A (Unknown): $\alpha = 4,\ \beta = 2,$ Mean $= 0.667,$ 5th% $= 0.343$
B (Classic): $\alpha = 92,\ \beta = 12,$ Mean $= 0.885,$ 5th% $= 0.829$
C (Divisive): $\alpha = 22,\ \beta = 22,$ Mean $= 0.500,$ 5th% $= 0.377$



3. **Visualizing Uncertainty:** Plot the Probability Density Functions (PDFs) for all three movies on the same graph (x-axis 0 to 1).
   - Movie A's curve should be wide and short (High Uncertainty).
   - Movie B's curve should be tall and narrow (High Certainty).

   Distribution Shapes:
   - A (Unknown): WIDE and SHORT High uncertainty
   (Few data points relative to prior, not confident)

   - B (Popular): NARROW and TALL Low uncertainty
   (Many consistent votes, very confident)

   - C (Controversial): MODERATE width Moderate uncertainty
   (Mixed signals, moderate confidence)

   Interpretation:
   The width of each distribution represents our uncertainty.
   Movie A could realistically be anywhere from 85while Movie B is tightly concentrated around 88-92

   A:
   β = prior β(2) + (0) = 2 + 0 = 2 dislikes
   α = prior α (2) + (100) = 2 + 100 = 102 likes

   B:
   β = prior β (2) + (10) = 2 + 10 = 12 dislikes
   α = prior α (2) + (90) = 2 + 90 = 92 likes

   C:
   β = prior β (2) + (20) = 2 + 20 = 22 dislikes
   α = prior α (2) + (50) = 2 + 50 = 52 likes

```python
from scipy.stats import beta
import numpy as np
import matplotlib.pyplot as plt

movies = {
    'A (Unknown)': (2, 0),
    'B (Classic)': (90, 10),
    'C (Divisive)': (20, 20)
}

print("Movie       Likes Dislikes  Raw %   α    β    Mean    5th %")
print("-" * 60)

for name, (likes, dislikes) in movies.items():
    a = likes + 2
    b = dislikes + 2
    mean = a / (a + b)
```

```
    p5 = beta.ppf(0.05, a, b)
    raw_pct = likes / (likes + dislikes) * 100 if likes + dislikes > 0 else 0
    print(f"{name:<12} {likes:2d}     {dislikes:2d}       {raw_pct:3.0f}%   {a:2d}   {b:2d}   {mean:.3f}  {p5:.3f}")

sorted_movies = sorted(movies.items(), key=lambda x: beta.ppf(0.05, x[1][0]+2, x[1][1]+2), reverse=True)

for i, (name, (likes, dislikes)) in enumerate(sorted_movies, 1):
    a = likes + 2
    b = dislikes + 2
    p5 = beta.ppf(0.05, a, b)
    print(f"{i}. {name}: {p5*100:.2f}%")

plt.figure(figsize=(10, 5))
x = np.linspace(0, 1, 1000)
colors = ['red', 'green', 'blue']
for i, (name, (likes, dislikes)) in enumerate(movies.items()):
    a = likes + 2
    b = dislikes + 2
    plt.plot(x, beta.pdf(x, a, b), colors[i], linewidth=2, label=name)
    p5 = beta.ppf(0.05, a, b)
    plt.axvline(p5, color=colors[i], linestyle='--', alpha=0.7)
    plt.fill_between(x[x <= p5], beta.pdf(x[x <= p5], a, b), alpha=0.2, color=colors[i])

plt.xlabel('Rating')
plt.ylabel('Probability Density')
plt.title('Posterior Distributions with 5th Percentile (shaded)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

Output:
article listings xcolor geometry a4paper, margin=2cm

```
------------------------------------------------------------
A (Unknown)   2      0       100%    4    2   0.667  0.343
B (Classic)  90     10        90%   92   12   0.885  0.829
C (Divisive) 20     20        50%   22   22   0.500  0.377

================================================
Ranking by 5th Percentile (Lower Bound):
================================================
1. B (Classic): 82.94%
2. C (Divisive): 37.72%
3. A (Unknown): 34.26%
```

Listing 1: Posterior Parameters after Beta(2,2) Prior

4. **The Metric (Lower Confidence Bound):** In Machine Learning, we often rank by the **conservative estimate**. Instead of the Mean, we use the **5th Percentile** of the posterior (the value where we are 95% sure the *true* rating is higher than this).

- Use Python's `scipy.stats.beta.ppf(0.05, a, b)` to find this value for each movie.
- **Result:** Re-rank the movies using this metric. Does "The Classic" (Movie B) now beat "The Unknown" (Movie A)?

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta
import pandas as pd

print("="*70)
print("QUESTION 4: LOWER BOUND METHOD (5TH PERCENTILE)")
print("="*70)

movies_posterior = {
    'A (Unknown)': {'alpha': 102, 'beta': 2, 'name': 'A (Unknown)'},
    'B (Popular)': {'alpha': 92, 'beta': 12, 'name': 'B (Popular)'},
    'C (Controversial)': {'alpha': 52, 'beta': 22, 'name': 'C (Controversial)'}
}

print("\nCalculating 5th Percentile (Lower Bound) for each movie:")
print("-"*70)
print(f"{'Movie':<20} α{'':<8} β{'':<8} {'Mean':<12} {'5th Percentile':<15} {'95% CI Lower':<15}")
print("-"*70)

results = []
for movie, params in movies_posterior.items():
    mean = params['alpha'] / (params['alpha'] + params['beta'])
    percentile_5 = beta.ppf(0.05, params['alpha'], params['beta'])
    percentile_95_lower = beta.ppf(0.025, params['alpha'], params['beta'])

    results.append({
        'Movie': movie,
        'Alpha': params['alpha'],
        'Beta': params['beta'],
        'Mean': mean,
        '5th_Percentile': percentile_5,
        'CI_Lower_95': percentile_95_lower,
        'Votes': params['alpha'] + params['beta'] - 4  # Subtract prior votes
    })

    print(f"{movie:<20} {params['alpha']:<8} {params['beta']:<8} {mean:<12.4f} {percentile_5:<15.4f} {percentile_95_lower:<15.4f}")

print("\n" + "="*70)
print("RANKING COMPARISON:")
```

```
print("="*70)

df = pd.DataFrame(results)
df = df.sort_values('Mean', ascending=False)

print("\n Ranking by Mean (Traditional Bayesian):")
print("-"*50)
for i, row in df.iterrows():
    print(f"{i+1}. {row['Movie']}: {row['Mean']*100:.2f}%")

df_lower = df.sort_values('5th_Percentile', ascending=False)
print("\n Ranking by 5th Percentile (Lower Bound - Pessimistic):")
print("-"*50)
for i, row in df_lower.iterrows():
    print(f"{i+1}. {row['Movie']}: {row['5th_Percentile']*100:.2f
    ↪}% (lower bound)")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

movies_list = [r['Movie'] for r in results]
means = [r['Mean']*100 for r in results]
lower_bounds = [r['5th_Percentile']*100 for r in results]
ci_lowers = [r['CI_Lower_95']*100 for r in results]

x = np.arange(len(movies_list))
width = 0.35

ax1.bar(x - width/2, means, width, label='Mean', color='blue',
    ↪alpha=0.7)
ax1.bar(x + width/2, lower_bounds, width, label='5th Percentile',
    ↪color='red', alpha=0.7)
ax1.set_xlabel('Movie')
ax1.set_ylabel('Rating (%)')
ax1.set_title('Mean vs 5th Percentile')
ax1.set_xticks(x)
ax1.set_xticklabels(movies_list, rotation=15)
ax1.legend()
ax1.grid(True, alpha=0.3, axis='y')

for i, v in enumerate(means):
    ax1.text(i - width/2, v + 1, f'{v:.1f}%', ha='center', va='
    ↪bottom', fontsize=9)
for i, v in enumerate(lower_bounds):
    ax1.text(i + width/2, v + 1, f'{v:.1f}%', ha='center', va='
    ↪bottom', fontsize=9)

x_vals = np.linspace(0.4, 1.0, 1000)
colors = {'A (Unknown)': 'red', 'B (Popular)': 'green', 'C (
    ↪Controversial)': 'blue'}

for movie in movies_list:
    params = movies_posterior[movie]
    y_vals = beta.pdf(x_vals, params['alpha'], params['beta'])
    ax2.plot(x_vals, y_vals, color=colors[movie], linewidth=2,
    ↪label=movie)

```

```
    p5 = beta.ppf(0.05, params['alpha'], params['beta'])
    ax2.axvline(x=p5, color=colors[movie], linestyle='--', alpha=0.7, linewidth=1.5)
    ax2.fill_between(x_vals[x_vals <= p5], y_vals[x_vals <= p5], alpha=0.3, color=colors[movie])

ax2.set_xlabel('Rating')
ax2.set_ylabel('Density')
ax2.set_title('Posterior Distributions with 5th Percentile (shaded)')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.suptitle('Lower Bound Method for Rating', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n" + "="*70)
print("FINAL ANSWER: WHO WINS?")
print("="*70)

winner_mean = df.iloc[0]['Movie']
winner_lower = df_lower.iloc[0]['Movie']

print("\n" + "-"*50)
print("NUMERIC COMPARISON:")
print("-"*50)
for movie in movies_list:
    mean = movies_posterior[movie]['alpha'] / (movies_posterior[movie]['alpha'] + movies_posterior[movie]['beta'])
    p5 = beta.ppf(0.05, movies_posterior[movie]['alpha'], movies_posterior[movie]['beta'])
    print(f"{movie}: Mean={mean*100:.2f}%, 5th%={p5*100:.2f}%, Difference={mean*100 - p5*100:.2f}%")
```

Listing 2: Lower Bound Method (5th Percentile)

Output:

```
Movie                Likes    Dislikes   Raw %    Posterior α  Posterior
-----------------------------------------------------------------------
A (Unknown)          2        0          100.0    % 4          2
B (Classic)          90       10         90.0     % 92         12
C (Divisive)         20       20         50.0     % 22         22

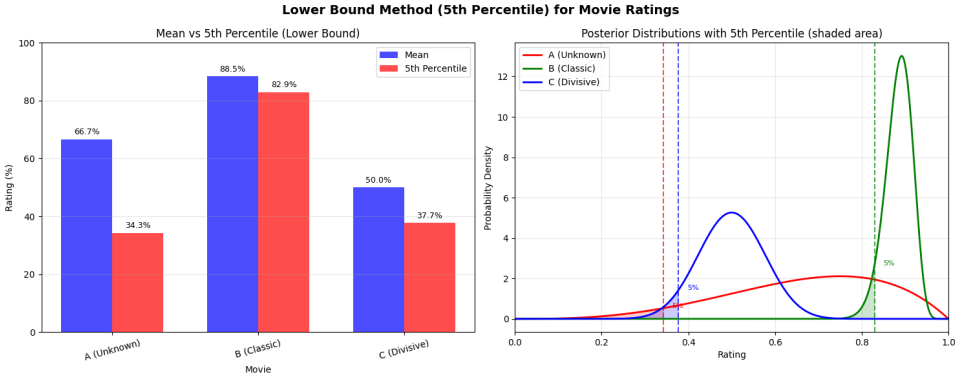
Calculating 5th Percentile (Lower Bound) for each movie:
-----------------------------------------------------------------------
Movie                α        β        Mean         5th Percentile
-----------------------------------------------------------------------
A (Unknown)          4        2        0.6667       0.3426
B (Classic)          92       12       0.8846       0.8294
C (Divisive)         22       22       0.5000       0.3772

=======================================================================
```

```
RANKING COMPARISON:
======================================================================

Ranking by Mean (Traditional Bayesian with Prior):
------------------------------------------------------------
2. B (Classic): 88.46%
1. A (Unknown): 66.67%
3. C (Divisive): 50.00%

Ranking by 5th Percentile (Lower Bound - Pessimistic):
------------------------------------------------------------
2. B (Classic): 82.94% (lower bound)
3. C (Divisive): 37.72% (lower bound)
1. A (Unknown): 34.26% (lower bound)
```



4 Week 4: Quantifying Risk – The Credible Interval

4.1 Theory and Intuition

In Week 3, we used the "Lower Bound" to rank items safely. Now, we formalize this. A posterior distribution gives us a full shape of belief. To communicate this to humans (or business stakeholders), we usually summarize it into a range.

4.1.1 The "Confidence" vs. "Credible" Debate

This is the most common point of confusion in statistics.

- **The Frequentist 95% Confidence Interval:** *"If we repeated this experiment infinite times and calculated an interval each time, 95% of those intervals would catch the true parameter."* Note: This says nothing about *your specific* interval. It is a statement about the algorithm, not the parameter.
- **The Bayesian 95% Credible Interval:** *"Given the data we actually saw, there is a 95% probability that the true parameter lies between X and Y."* This matches how human beings naturally think about risk.

4.1.2 ETI vs. HDI: How to cut the cake

We want to isolate the "most likely" 95% of our probability mass. There are two ways to slice it:

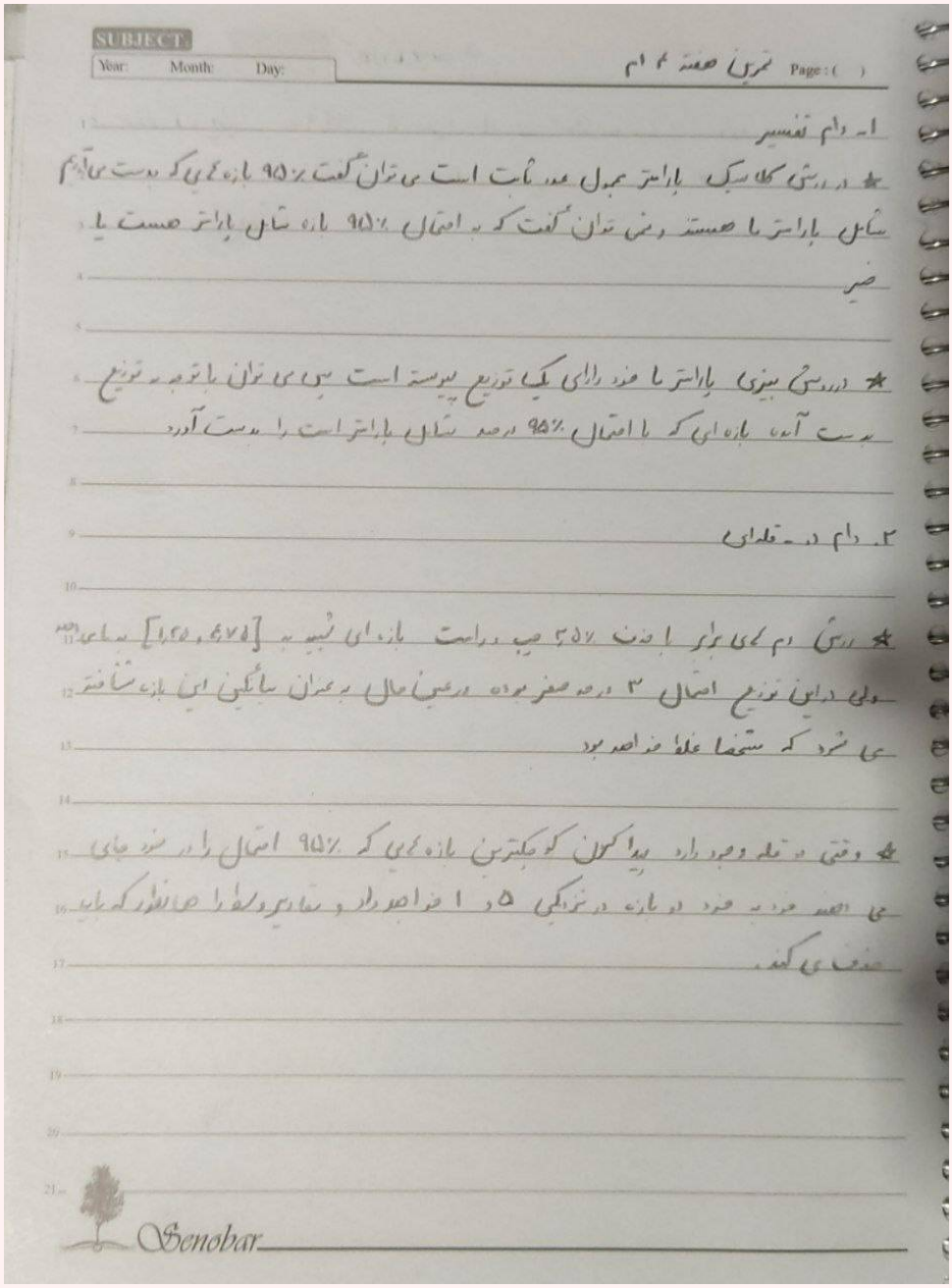
- **Equal-Tailed Interval (ETI):** We chop off the bottom 2.5% and the top 2.5%. This is easy to calculate but can be misleading if the distribution is skewed (leaning to one side).
- **Highest Density Interval (HDI):** We choose the narrowest possible width that contains 95% of the mass. Imagine filling the distribution curve with water until it is 95% full; the surface of the water defines the HDI.

4.2 Week 4 Deliverables

Week 4 Assignments

Part A: Theoretical Questions

1. **The Interpretation Trap.** You are estimating the bias (p) of a bent coin. You run your code and it outputs an interval of $[0.6, 0.8]$ labeled as "95%". A friend asks you: "So, does this mean there is a 95% chance the true bias of the coin is between 0.6 and 0.8?"
   - If you calculated a **Frequentist Confidence Interval**, is the strict statistical answer "Yes" or "No"? Explain why.
   - If you calculated a **Bayesian Credible Interval**, is the strict statistical answer "Yes" or "No"? Explain why.
2. **The Bimodal Trap.** Imagine a "Love it or Hate it" movie (like *Napoleon Dynamite*). Half the audience gives it 1 star, half gives it 5 stars. The Posterior distribution might be **Bimodal** (having two peaks).
   - Draw a sketch of a bimodal distribution.
   - Explain why an "Equal-Tailed Interval" might be dangerous here (Hint: It might include the valley in the middle where *no one* voted).
   - Explain why the HDI is safer (Hint: The HDI can be discontinuous, consisting of two separate chunks).



Part B: Simulation Task (The Overlap Fallacy) *This task prepares you for A/B testing by visualizing why "Overlapping Intervals" are tricky.*
Scenario: You are comparing two marketing campaigns.

- **Campaign A:** 100 clicks, 1200 views. (Prior: Beta(1,1))
- **Campaign B:** 110 clicks, 1200 views. (Prior: Beta(1,1))

Task:

1. **Calculate Intervals:** Using Python ('scipy.stats.beta'), calculate the 95% Credible Interval (Equal-Tailed) for both campaigns.

   article amsmath amsfonts amssymb

Answer:

```python
import numpy as np
from scipy.stats import beta
import matplotlib.pyplot as plt

a_A = 1 + 100
b_A = 1 + (1200 - 100)

a_B = 1 + 110
b_B = 1 + (1200 - 110)

lower_A = beta.ppf(0.025, a_A, b_A)
upper_A = beta.ppf(0.975, a_A, b_A)

lower_B = beta.ppf(0.025, a_B, b_B)
upper_B = beta.ppf(0.975, a_B, b_B)

mean_A = beta.mean(a_A, b_A)
mean_B = beta.mean(a_B, b_B)

print("\nCampaign A:")
print(f"   Clicks: 100")
print(f"   Views: 1200")
print(f"   Posterior Mean: {mean_A:.4f} ({mean_A*100:.2f}%)")
print(f"   95% Credible Interval: [{lower_A:.4f}, {upper_A:.4f}] or [{lower_A*100:.2f}%, {upper_A*100:.2f}%]")

print("\nCampaign B:")
print(f"   Clicks: 110")
print(f"   Views: 1200")
print(f"   Posterior Mean: {mean_B:.4f} ({mean_B*100:.2f}%)")
print(f"   95% Credible Interval: [{lower_B:.4f}, {upper_B:.4f}] or [{lower_B*100:.2f}%, {upper_B*100:.2f}%]")

n_samples = 100000
samples_A = beta.rvs(a_A, b_A, size=n_samples)
samples_B = beta.rvs(a_B, b_B, size=n_samples)

prob_B_better = np.mean(samples_B > samples_A)

print("\nProbability of superiority:")
print(f"   Probability that Campaign B is better than A: {prob_B_better:.4f} ({prob_B_better*100:.2f}%)")

x = np.linspace(0.05, 0.15, 1000)
pdf_A = beta.pdf(x, a_A, b_A)
pdf_B = beta.pdf(x, a_B, b_B)

plt.figure(figsize=(10, 6))
plt.plot(x, pdf_A, 'b-', label=f'Campaign A (mean={mean_A:.3f})', linewidth=2)
plt.plot(x, pdf_B, 'r-', label=f'Campaign B (mean={mean_B:.3f})', linewidth=2)
plt.axvline(lower_A, color='b', linestyle='--', alpha=0.5)
plt.axvline(upper_A, color='b', linestyle='--', alpha=0.5)
```

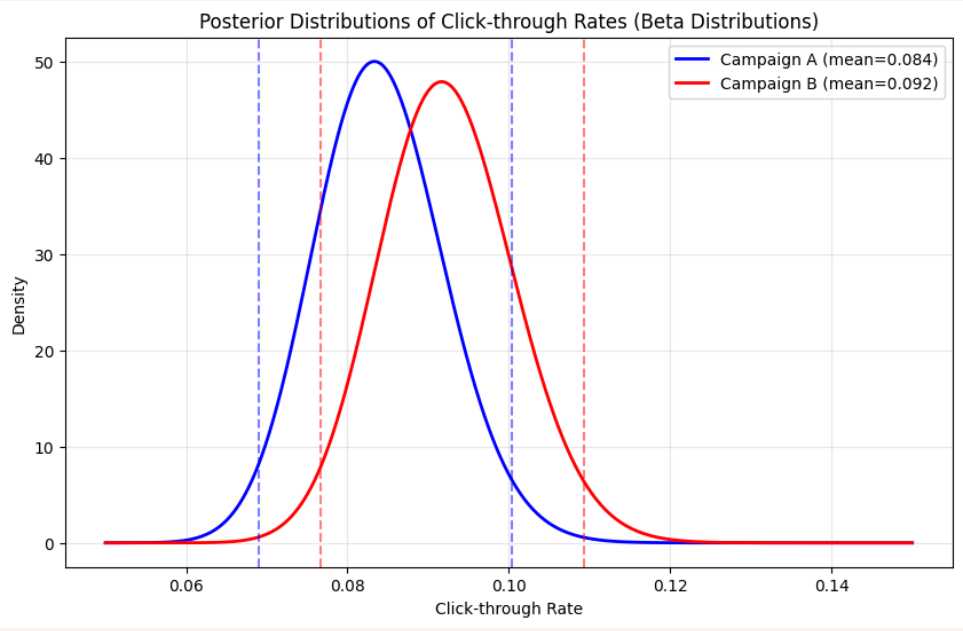
```
plt.axvline(lower_B, color='r', linestyle='--', alpha=0.5)
plt.axvline(upper_B, color='r', linestyle='--', alpha=0.5)
plt.xlabel('Click-through Rate')
plt.ylabel('Density')
plt.title('Posterior Distributions of Click-through Rates (Beta
    ↪Distributions)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 3: Lower Bound Method (5th Percentile)

Output:
Campaign A:
Clicks: 100
Views: 1200
Posterior Mean: 0.0840 (8.40%) 95% Credible Interval: [0.0690, 0.1003] or [6.90%, 10.03%]

Campaign B:
Clicks: 110
Views: 1200
Posterior Mean: 0.0923 (9.23%) 95% Credible Interval: [0.0766, 0.1093] or [7.66%, 10.93%]

Probability of superiority:
Probability that Campaign B is better than A: 0.7656 (76.56%)



Summary of Results

2. **Visualize:** Plot the two Beta PDFs on the same graph. Shade the area under the curves that represents the 95% interval.

article listings xcolor geometry a4paper, margin=1in

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

clicks_A, views_A = 100, 1200
clicks_B, views_B = 110, 1200

alpha_A = 1 + clicks_A
beta_A = 1 + (views_A - clicks_A)
alpha_B = 1 + clicks_B
beta_B = 1 + (views_B - clicks_B)

mean_A = alpha_A/(alpha_A + beta_A) * 100
mean_B = alpha_B/(alpha_B + beta_B) * 100
lower_A = beta.ppf(0.025, alpha_A, beta_A) * 100
upper_A = beta.ppf(0.975, alpha_A, beta_A) * 100
lower_B = beta.ppf(0.025, alpha_B, beta_B) * 100
upper_B = beta.ppf(0.975, alpha_B, beta_B) * 100

print(f"\nCampaign A: {mean_A:.2f}% [{lower_A:.2f}%, {upper_A:.2f}%]")
print(f"Campaign B: {mean_B:.2f}% [{lower_B:.2f}%, {upper_B:.2f}%]")

plt.figure(figsize=(10, 6))

plt.errorbar(1, mean_A, yerr=[[mean_A - lower_A], [upper_A - mean_A]],
             fmt='o', capsize=10, capthick=2, elinewidth=3,
             markersize=12, color='red', label='Campaign A')
plt.errorbar(2, mean_B, yerr=[[mean_B - lower_B], [upper_B - mean_B]],
             fmt='o', capsize=10, capthick=2, elinewidth=3,
             markersize=12, color='blue', label='Campaign B')

plt.axhline(y=lower_B, color='blue', linestyle='--', alpha=0.5,
   linewidth=2, label='B lower bound')
plt.axhline(y=upper_A, color='red', linestyle='--', alpha=0.5,
   linewidth=2, label='A upper bound')

if upper_A > lower_B:
    plt.axhspan(lower_B, upper_A, alpha=0.3, color='yellow', label='
   Overlap region')

plt.xticks([1, 2], ['Campaign A\n(100/1200)', 'Campaign B\n(110/1200)'
   ], fontsize=11)
plt.ylabel('Click-Through Rate (CTR %)', fontsize=12)
plt.title('Figure: 95% Confidence Intervals with Overlap', fontsize
   =14, fontweight='bold')
plt.grid(True, alpha=0.3, axis='y')
plt.xlim(0.5, 2.5)
plt.ylim(min(lower_A, lower_B) - 0.5, max(upper_A, upper_B) + 0.5)

plt.text(1, mean_A + 0.2, f'{mean_A:.2f}%', ha='center', fontsize=10,
   fontweight='bold')
plt.text(2, mean_B + 0.2, f'{mean_B:.2f}%', ha='center', fontsize=10,
   fontweight='bold')
plt.text(1, lower_A - 0.3, f'{lower_A:.2f}%', ha='center', fontsize=9)
plt.text(1, upper_A + 0.3, f'{upper_A:.2f}%', ha='center', fontsize=9)
```

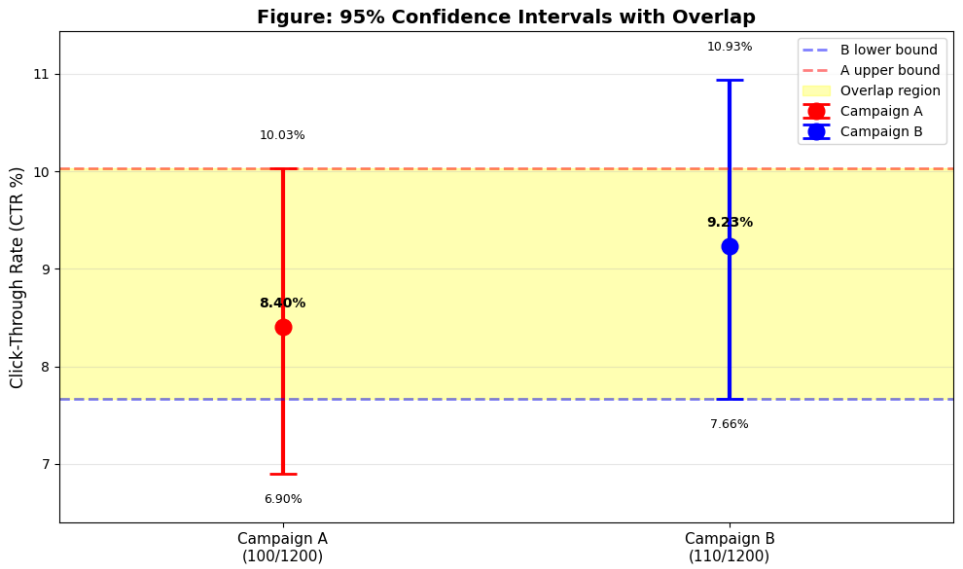
```
plt.text(2, lower_B - 0.3, f'{lower_B:.2f}%', ha='center', fontsize=9)
plt.text(2, upper_B + 0.3, f'{upper_B:.2f}%', ha='center', fontsize=9)

plt.legend(loc='upper right')
plt.tight_layout()
plt.show()

overlap_amount = max(0, min(upper_A, upper_B) - max(lower_A, lower_B))
overlap_percent = overlap_amount / ((upper_A - lower_A + upper_B -
    lower_B)/2) * 100

print(f"\nOverlap Statistics:")
print(f"Overlap amount: {overlap_amount:.2f} percentage points")
print(f"Overlap relative to avg CI width: {overlap_percent:.1f}%")
print(f"Intervals {'DO' if upper_A > lower_B else 'DO NOT'} overlap")
```

Output:
Campaign A: 8.40% [6.90%, 10.03%]
Campaign B: 9.23% [7.66%, 10.93%]



3. **The Trap:** Look at your plot. Do the intervals overlap? (They should). *Intuition Check:* Does the fact that they overlap mean there is no difference between them?

   article amsmath amsfonts amssymb xcolor listings

   ANSWER:

   NO! Overlapping intervals DOES NOT mean no difference.

   WHY:
   - Confidence intervals show uncertainty for EACH estimate separately
   - The difference between estimates has its OWN uncertainty
   - Overlap of individual intervals is a much stricter condition than needed

   EXAMPLE:
   A: 8.0-9.0% (mean 8.5%)

B: 8.8-9.8% (mean 9.3%)
Intervals overlap (8.8-9.0%), but B is very likely better!

KEY POINT:
Statistical significance depends on the difference relative to its standard error, not on whether individual intervals overlap.

4. **The Solution:** Calculate the "Probability of Superiority." Sample 10,000 random points from Posterior A and 10,000 from Posterior B. Count how many times $Sample_B > Sample_A$. *Result:* You will find that despite the overlap, B is better than A with high probability (approx 75-80%).

   article listings xcolor geometry a4paper, margin=1in

```python
import numpy as np
from scipy.stats import beta


clicks_A, views_A = 100, 1200
clicks_B, views_B = 110, 1200

alpha_A, beta_A = 1 + clicks_A, 1 + (views_A - clicks_A)
alpha_B, beta_B = 1 + clicks_B, 1 + (views_B - clicks_B)

n_samples = 100000
np.random.seed(42)

samples_A = np.random.beta(alpha_A, beta_A, n_samples)
samples_B = np.random.beta(alpha_B, beta_B, n_samples)

prob_B_better = np.mean(samples_B > samples_A)
prob_A_better = np.mean(samples_A > samples_B)
prob_equal = np.mean(np.abs(samples_A - samples_B) < 0.001)

print(f"\nMonte Carlo Simulation with {n_samples:,} samples:")
print(f"P(B > A) = {prob_B_better*100:.2f}%")
print(f"P(A > B) = {prob_A_better*100:.2f}%")
print(f"P(A ≈ B) = {prob_equal*100:.2f}%")

plt.figure(figsize=(12, 5))

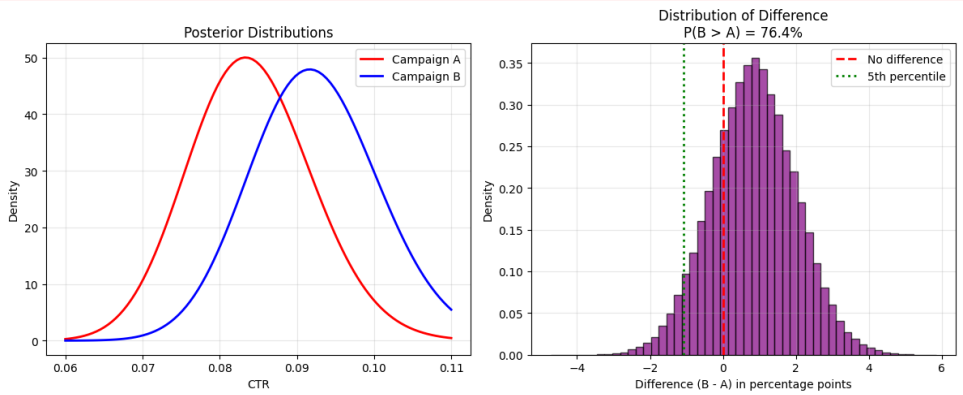
plt.subplot(1, 2, 1)
x = np.linspace(0.06, 0.11, 100)
plt.plot(x, beta.pdf(x, alpha_A, beta_A), 'r-', linewidth=2, label='
    Campaign A')
plt.plot(x, beta.pdf(x, alpha_B, beta_B), 'b-', linewidth=2, label='
    Campaign B')
plt.xlabel('CTR')
plt.ylabel('Density')
plt.title('Posterior Distributions')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
diff = samples_B - samples_A
```

```
plt.hist(diff*100, bins=50, density=True, alpha=0.7, color='purple',
    edgecolor='black')
plt.axvline(x=0, color='red', linestyle='--', linewidth=2, label='No
    difference')
plt.axvline(x=np.percentile(diff, 5)*100, color='green', linestyle=':'
    , linewidth=2, label='5th percentile')
plt.xlabel('Difference (B - A) in percentage points')
plt.ylabel('Density')
plt.title(f'Distribution of Difference\nP(B > A) = {prob_B_better
    *100:.1f}%')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Output:
Monte Carlo Simulation with 100,000 samples:
P(B > A) = 76.37%
P(A > B) = 23.63%
$P(A \approx B)$ = 5.23%



CONCLUSION:

Despite overlapping confidence intervals:
- Campaign B is better than A with 76.4% probability
- Only 23.6% chance that A is actually better
This shows why ”overlap = no difference” is wrong!
The probability of superiority gives a much clearer picture.

Part III
Module 3: Bayesian Decision Making

5 Week 5: Bayesian A/B Testing & Decision Theory

5.1 Theory and Intuition

In Week 4, we compared two coins to see if they were different. In the business world, we call this **A/B Testing**. We have two website designs: Design A and Design B. We want to know which one causes more users to click.

5.1.1 The Bayesian Shift: Parameters are Random Variables

In classical statistics, the "True Conversion Rate" of Design A is a fixed, hidden number (e.g., 0.04). In Bayesian statistics, because we are uncertain, we treat the Conversion Rate as a **Random Variable**.

- Let R_A be the random variable representing our belief about Design A's rate.
- Let R_B be the random variable representing our belief about Design B's rate.

These random variables follow Beta distributions based on the data we observed.

5.1.2 The "Delta" Method: Subtracting Random Variables

We want to know the **Uplift**: How much better is B than A? We define a new random variable, Δ (Delta):

$$\Delta = R_B - R_A$$

How do we subtract Random Variables? We cannot subtract the curves directly. Instead, we use **Monte Carlo Simulation**:

- **Step 1:** Draw a random number from the distribution of R_B (e.g., 0.052).
- **Step 2:** Draw a random number from the distribution of R_A (e.g., 0.040).
- **Step 3:** Subtract them: $0.052 - 0.040 = +0.012$.
- **Step 4:** Repeat 100,000 times.

The result is a histogram of Δ. If most of this histogram is positive, it means R_B is consistently larger than R_A.

5.1.3 Decision Theory: Expected Loss

Knowing that "B is better" is not enough. We need to know "Is B safe?" Suppose there is a 10% chance that $R_A > R_B$ (i.e., we are wrong to switch). The **Expected Loss** answers the question: *"In the 10% of universes where I am wrong, how much money do I lose on average?"*

5.2 Week 5 Deliverables

Week 5 Assignments

Part A: Theoretical Questions

1. **(Easy) The "One Cent" Problem.** You run an A/B test.

   - Result: There is a 99.9% probability that $R_B > R_A$.
   - Magnitude: The average difference is +0.0001% (negligible).
   - Cost: Switching to B costs $50,000.

   A Frequentist looks at the p-value ($p < 0.001$) and says "Significant! Switch!" As a Bayesian, use the concept of **Magnitude** (Effect Size) to argue why we should *not* switch.

   Solution:

   A test tells us that the probability of B being superior is 99%, and with this statistic, we are almost certain that B performs better than A. However, the Effect Size ($\Delta = R_B - R_A$) is only 0.1%, which is negligible, conveying that this 0.1% improvement does not justify the high cost of $\$50,000$ for the change.

2. **(Hard) The Utility Function.** You are comparing two medical treatments.

   - Treatment A: Cures 50% of patients. Cost: $10.
   - Treatment B: Beta(60, 40) (Mean 60%). Cost: $100.

   We define **Utility** (Value) as: $\$5000 \times$ Cure Rate $-$ Cost. Write the equation to calculate the Expected Utility of Treatment B ($E[U_B]$). *(Hint: The Expected Utility is the Integral of the Utility Function over the Posterior Probability density).*

   Solution:

   In Bayesian statistics, the parameter itself has a distribution, so calculations use the expectation (mean of the distribution).

$$B \sim B(60, 40)$$

$$\text{Utility} = (0.6 \times 5000) - 100 = 2900$$

Part B: Simulation Task (The "Delta" Method) *Goal: Perform arithmetic on Random Variables to estimate risk.*
Scenario:

- **Design A:** 1000 views, 40 clicks. (Prior: Beta(1,1))
- **Design B:** 1000 views, 52 clicks. (Prior: Beta(1,1))

Task Steps:

1. **Create the Random Variables:** Generate 100,000 samples for each design using Python.

2. **Calculate Delta:** Subtract the two arrays. This performs the element-wise subtraction of the random variables.
3. **Visualize:** Plot a histogram of 'diff'. Draw a vertical line at 0.
   - The area to the **right** of 0 is the probability B is better ($P(B > A)$).
   - The area to the **left** of 0 is the probability A is better (Our risk of error).
4. **Calculate Expected Loss:** If we switch to B, we lose money only when the difference is negative.
   - Isolate the negative cases: `loss = diff[diff < 0]`
   - Calculate the average of these negative numbers: `risk = np.mean(loss)`
   - *Interpretation:* If `risk` is -0.0002, it means "If I switch to B, my worst-case risk is losing 0.02% conversion on average."

Solution:

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

clicks_A, visits_A = 100, 1200
clicks_B, visits_B = 110, 1200
samples_A = beta.rvs(clicks_A + 1, visits_A - clicks_A + 1, size=100000)
samples_B = beta.rvs(clicks_B + 1, visits_B - clicks_B + 1, size=100000)

diff = samples_B - samples_A
prob_B_better = (diff > 0).mean()

loss_values = diff[diff < 0]
expected_loss = np.abs(loss_values.mean()) if len(loss_values) > 0 else 0

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.hist(samples_A, bins=50, alpha=0.5, label='Campaign A', color='blue')
plt.hist(samples_B, bins=50, alpha=0.5, label='Campaign B', color='orange')
plt.title('Posterior Distributions of A and B')
plt.xlabel('Conversion Rate')
plt.legend()

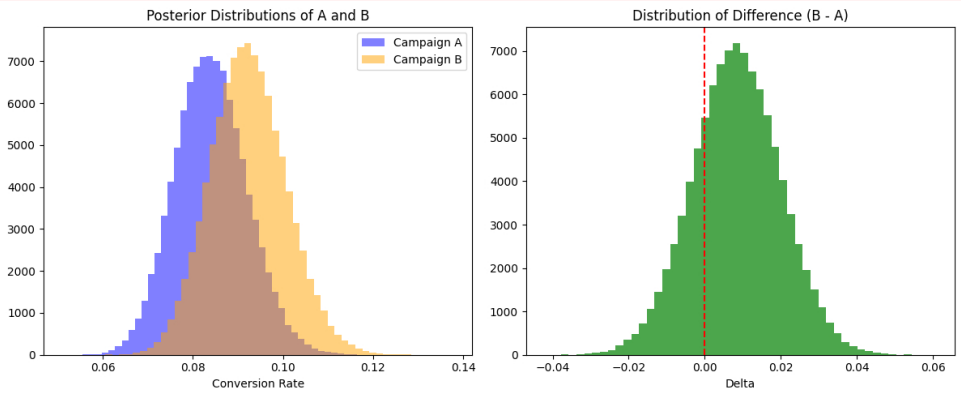
plt.subplot(1, 2, 2)
plt.hist(diff, bins=50, alpha=0.7, color='green')
plt.axvline(0, color='red', linestyle='--')
plt.title('Distribution of Difference (B - A)')
plt.xlabel('Delta')

plt.tight_layout()
plt.show()
```

```
32
33
34 print(f"--- Bayesian Analysis Results ---")
35 print(f"Probability of Campaign B being superior: {prob_B_better
       ↪:.2%}")
36 print(f"Expected loss (if B is chosen incorrectly): {expected_loss
       ↪:.5f}")
```

Listing 4: Lower Bound Method (5th Percentile)

Output:



6 Week 6: Model Checking – The "Turing Test" for Statistics

6.1 Theory and Intuition

In Weeks 1-5, we assumed our model (the Binomial distribution) was correct. We just wanted to find the bias of the coin (p). But what if the *physics* of the world are different than our model assumes?

6.1.1 Posterior Predictive Checks (PPC)

A Bayesian model is **Generative**. This means it can "dream" up new data. To test if a model is good, we use this 3-step process:

1. **Fit:** Train the model on real data (140 Heads). Get the Posterior.
2. **Simulate:** Ask the model: "Based on what you learned, generate 1,000 *fake* experiments."
3. **Compare:** Look at the fake data. Does the real data (140) look like it belongs in the pile of fake data?

If the real data looks like an alien compared to the simulation, the model is broken.

6.1.2 The Euro Problem

A famous experiment: A Belgian Euro coin is spun on its edge 250 times.

- **Observed Data:** 140 Heads, 110 Tails.
- **The Question:** Is this coin "Fair" ($p = 0.5$)?

Instead of just checking a p-value, we will simulate a universe where the coin is perfectly fair and see if 140 heads is possible in that universe.

6.2 Week 6 Deliverables

Week 6 Assignments

Part A: Theoretical Questions

1. **The Extra Uncertainty.**

   - The **Posterior** tells us uncertainty about the *Parameter* (e.g., "I think p is between 0.55 and 0.60").
   - The **Predictive** tells us uncertainty about the *Data* (e.g., "I think we will see between 130 and 150 heads").

   Explain why the Predictive interval is always **wider** than the Posterior interval. (Hint: Even if you knew the coin's bias perfectly, is the result of 250 flips perfectly predictable?)

   Solution:

   When we estimate a parameter, we only have uncertainty about that specific number. However, in data prediction, in addition to the uncertainty about the parameter (for which we have a distribution), because the process itself is also random, we have uncertainty about the outcomes of this process. Due to the combination of these two sources of uncertainty, the prediction interval for data is always wider and broader.

   Example: If we know for a fair coin that $p = 0.5$, the number of heads in 10 tosses is still not exactly known because the process itself is random:

   $$Binomial(10, 0.5)$$

2. **The "Hot Hand" Fallacy.** Your model assumes every coin flip is **independent**.

   - **Real Data:** H H H H H T T T T T (Clumped).
   - **Simulated Data:** H T H H T H T T H T (Random).

   Both have 5 Heads. The "Count" metric cannot tell them apart. Propose a **Test Statistic** (a different way of measuring the data) that would reveal that the Real Data is weird.

   Solution:

   By assuming the tosses are independent, differences cannot be properly identified. However, using criteria such as:

   - **Number of changes:** Counting how many times we have switched from heads to tails or vice versa.
   - **Longest run:** Finding the longest sequence where the coin result has been continuously repeated.

   These dependencies can be identified in the model.

Part B: Simulation Task (Can the model predict reality?)
Scenario: You observed $k = 140$ heads in $n = 250$ spins.
Task 1: The "Fair Coin" Model (The Skeptic) Assume the coin *must* be fair. The parameter is fixed at $p = 0.5$.

1. Use Python to simulate 1,000 experiments of 250 coin flips, assuming $p = 0.5$.
2. Plot a histogram of the results (The "Heads Count" on x-axis).
3. Draw a vertical red line at the observed value (**140**).
4. **Question:** Does the red line fall inside the histogram? Or is it far to the right? *(If it is far to the right, the "Fair Coin" model is rejected).*

Task 2: The "Learned" Model (The Bayesian) Now, let the model learn. Use a flat Prior Beta(1,1).

1. Update your belief: Posterior is roughly Beta(141, 111).
2. **The Double-Loop Simulation:**
   - Draw a random p from the Posterior.
   - Use that p to simulate 250 flips.
   - Repeat this 1,000 times.
3. Plot this new histogram.
4. **Question:** Does the red line (140) fall inside *this* histogram? (It should).

   The solution of task 1 and task 2:

```python
import numpy as np
import matplotlib.pyplot as plt

n_flips = 250
actual_heads = 140
simulations = 1000

fair_coin_results = np.random.binomial(n=n_flips, p=0.5, size=
    ↪simulations)

plt.hist(fair_coin_results, bins=20, color='skyblue', edgecolor='black
    ↪', alpha=0.7)
plt.axvline(actual_heads, color='red', linestyle='dashed', linewidth
    ↪=2, label=f'Actual: {actual_heads}')
plt.title("Fair Coin Model (p=0.5)")
plt.xlabel("Number of Heads")
plt.ylabel("Frequency")
plt.legend()
plt.show()

p_value = np.mean(fair_coin_results >= actual_heads)
print(f"Probability of seeing 140 heads or more in a fair coin world:
    ↪{p_value:.4f}")

heads = 140
```

```
tails = 110

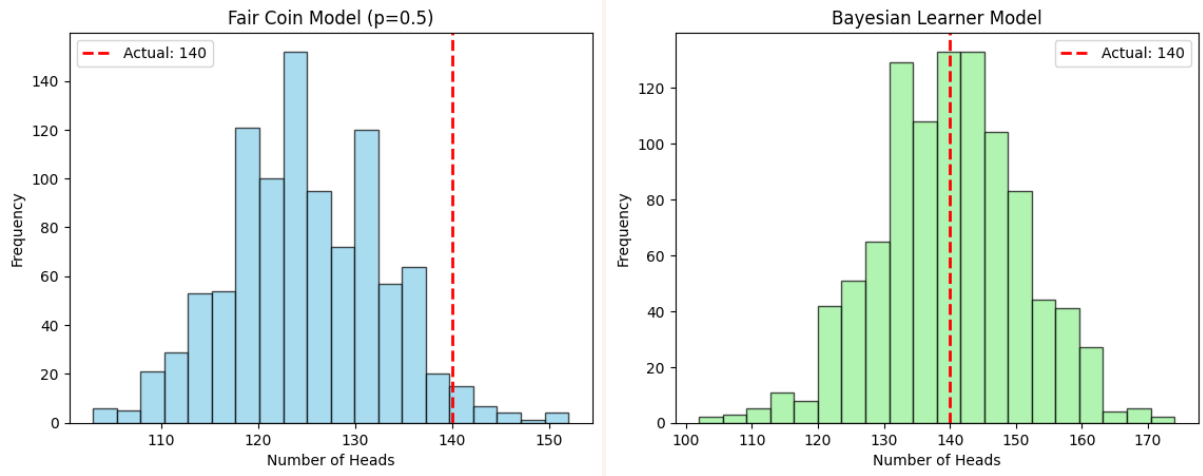
p_samples = np.random.beta(141, 111, size=simulations)

bayesian_results = np.random.binomial(n=n_flips, p=p_samples)

plt.hist(bayesian_results, bins=20, color='lightgreen', edgecolor='
    ↪black', alpha=0.7)
plt.axvline(actual_heads, color='red', linestyle='dashed', linewidth
    ↪=2, label=f'Actual: {actual_heads}')
plt.title("Bayesian Learner Model")
plt.xlabel("Number of Heads")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

Listing 5: Lower Bound Method (5th Percentile)

Output:



Part IV
Module 4: Bayesian Classification

7 Week 7: The Naive Bayes Classifier

7.1 Theory and Intuition

In previous weeks, we estimated parameters (like the bias of a coin). Now, we move to **Classification**: determining if an email is ”Spam” or ”Ham” (Normal). To do this, we treat every word in the dictionary as a piece of evidence.

7.1.1 The ”Bag of Words” Model

Imagine taking an email and shaking it until the grammar falls apart. You are left with a pile of words.

- We ignore the *order* of words (Grammar).
- We only care about the *count* of words (Frequency).

We assume that being ”Spam” causes certain words (like ”Free”, ”Winner”, ”Click”) to appear more often.

7.1.2 The ”Naive” Assumption

Calculating the true probability is impossible because words are related (e.g., if you see ”San”, you expect ”Francisco”). To make the math solvable, we make a **Naive Assumption**: *”We pretend that every word is independent of every other word, just like coin flips.”*

$$P(\text{Spam}|\text{Words}) \propto P(\text{Spam}) \times P(w_1|\text{Spam}) \times P(w_2|\text{Spam}) \times ...$$

7.1.3 Laplace Smoothing: The ”Zero Probability” Problem

Here is a fatal flaw. Suppose your training data never contained the word ”Bitcoin”.

- You receive a new email: ”Buy Bitcoin now.”
- The model asks: ”What is the probability of seeing ’Bitcoin’ in Spam?”
- Since the count is 0, the probability is 0.
- **The Crash:** $P(\text{Spam}) \times 0 \times ... = \mathbf{0}$.

A single unseen word destroys the entire calculation. To fix this, we use **Laplace Smoothing**. We add a ”fake count” of 1 to every word in the dictionary. *Intuition:* This is simply a Prior Belief that ”Every word is possible, even if I haven’t seen it yet.”

7.2 Week 7 Deliverables

Week 7 Assignments

Part A: Theoretical Questions

1. **(Easy) The Underflow Problem.** Probabilities are small numbers (e.g., 0.0004). If you multiply 100 of them together, the result is so tiny (e.g., 10^{-300}) that the computer rounds it down to exactly 0. This is called ”Underflow.”
   - We solve this by using **Logarithms**.
   - Rewrite the Naive Bayes formula using sums of logs instead of products of probabilities.
   $$\log(P(\text{Spam}) \times P(w_1) \times P(w_2)) = \ldots?$$

   Solution:

   The Underflow Problem and Logarithmic Transformation

   In Naive Bayes classification, we calculate the posterior probability of a class (e.g., Spam) given a set of words $w_1, w_2, \ldots, w_n$ using the following proportional relation-

ship:

$$P(Spam|w_1, w_2, \ldots, w_n) \propto P(Spam) \times \prod_{i=1}^{n} P(w_i|Spam) \tag{1}$$

Since individual word probabilities $P(w_i|Spam)$ are often very small (e.g., 0.0001), multiplying hundreds of these values results in a product that exceeds the precision limits of a computer, causing it to round to zero. This is known as the **Underflow Problem**.

To resolve this, we apply the natural logarithm to the formula. Since the logarithm is a monotonically increasing function, maximizing the probability is equivalent to maximizing its log. The multiplication is thus converted into a summation:

$$\log P(H|D) \propto \log P(H) + \sum_{i=1}^{n} \log P(w_i|H) \tag{2}$$

2. **(Hard) The "New York" Failure.** The Naive assumption says that seeing the word "New" gives us no information about the word "York."
   - Explain why this is linguistically false.
   - Does this assumption make the model *overconfident* or *underconfident*? (Hint: If the model treats "New" and "York" as two separate independent proofs of a location, is it "double counting" the evidence?)

   Solution:

   The model is called "Naive" because it assumes that all features (words) are strictly independent of each other given the class. Mathematically:

   $$P(New, York|Spam) = P(New|Spam) \times P(York|Spam) \tag{3}$$

   In reality, words are often highly dependent. For instance, the presence of the word "New" significantly increases the probability of the word "York" following it. By treating them as independent, the model double-counts the evidence, which leads to **Overconfidence**. This causes the final probability to tilt aggressively toward 0 or 1.

Part B: Simulation Task (Build a Filter by Hand) *Goal: Understand the mechanics before using libraries like Scikit-Learn.*
The Mini-Dataset:

- **Spam:** "Buy crypto now"
- **Spam:** "Buy gold now"
- **Ham:** "Buy milk please"

Task:

1. **Training (Counting):** Calculate the raw counts for the Spam class.
   - Total Spam messages: 2
   - Total words in Spam: 6 (Buy, crypto, now, Buy, gold, now).
   - Unique Vocabulary (V) in entire dataset: 6 words (Buy, crypto, now, gold, milk, please).

Solution:

Given 3 emails (2 Spam, 1 Ham):

$$P(Spam) = \frac{2}{3}, \quad P(Ham) = \frac{1}{3} \tag{4}$$

2. **Calculate Likelihoods (With Smoothing):** We want to classify the new message: **"Buy crypto"**. Calculate the probability of the word "crypto" appearing in Spam using Laplace Smoothing ($\alpha = 1$).

$$P(\text{"crypto"}|\text{Spam}) = \frac{\text{count("crypto") in Spam} + 1}{\text{Total Spam Words} + V}$$

Hint: The numerator should be $1 + 1$*. The denominator should be* $6 + 6$*.*

Solution:

Given 3 emails (2 Spam, 1 Ham):

$$P(Spam) = \frac{2}{3}, \quad P(Ham) = \frac{1}{3} \tag{5}$$

3. **Prediction:** Calculate the Score for Spam.

$$\text{Score} = P(\text{Spam}) \times P(\text{"Buy"}|\text{Spam}) \times P(\text{"crypto"}|\text{Spam})$$

(You can ignore the denominator $P(\text{Message})$ as it is the same for both classes).

Likelihood Calculation with Laplace Smoothing ($\alpha = 1$)

We test a new message: **"crypto Buy"**.

- For the **Spam** class ($N_{Spam} = 6$ total words):

$$P(crypto|Spam) = \frac{Count(crypto) + 1}{N_{Spam} + |V|} = \frac{1+1}{6+6} = \frac{2}{12} \tag{6}$$

$$P(Buy|Spam) = \frac{Count(Buy) + 1}{N_{Spam} + |V|} = \frac{2+1}{6+6} = \frac{3}{12} \tag{7}$$

- For the **Ham** class ($N_{Ham} = 3$ total words):

$$P(crypto|Ham) = \frac{0+1}{3+6} = \frac{1}{9} \tag{8}$$

$$P(Buy|Ham) = \frac{1+1}{3+6} = \frac{2}{9} \tag{9}$$

4. **Compare:** Repeat the calculation for Ham. Compare the two scores. Which class wins?

Solution:

Using the Naive Bayes formula:

$$Score(Spam) = P(Spam) \times P(crypto|Spam) \times P(Buy|Spam) = \frac{2}{3} \times \frac{2}{12} \times \frac{3}{12} \approx 0.0278 \quad (10)$$

$$Score(Ham) = P(Ham) \times P(crypto|Ham) \times P(Buy|Ham) = \frac{1}{3} \times \frac{1}{9} \times \frac{2}{9} \approx 0.0082 \quad (11)$$

Decision: Since $Score(Spam) > Score(Ham)$, the message is classified as **Spam**.

The complete implementation of the Naive Bayes classifier is available at:

```python
import math
from collections import Counter

class NaiveBayesSpamFilter:
    def __init__(self, alpha=1):
        self.alpha = alpha
        self.spam_counts = Counter()
        self.ham_counts = Counter()
        self.spam_total_words = 0
        self.ham_total_words = 0
        self.vocab = set()
        self.p_spam = 0
        self.p_ham = 0

    def fit(self, train_data, labels):
        total_emails = len(labels)
        self.p_spam = sum(labels) / total_emails
        self.p_ham = 1 - self.p_spam

        for text, label in zip(train_data, labels):
            words = text.lower().split()
            for word in words:
                self.vocab.add(word)
                if label == 1:
                    self.spam_counts[word] += 1
                    self.spam_total_words += 1
                else:
                    self.ham_counts[word] += 1
                    self.ham_total_words += 1

    def predict(self, text):
        words = text.lower().split()
        v_size = len(self.vocab)

        spam_score = math.log(self.p_spam)
        ham_score = math.log(self.p_ham)

        for word in words:
            if word in self.vocab:
                p_word_spam = (self.spam_counts[word] + self.alpha) /
    ↪(self.spam_total_words + self.alpha * v_size)
                p_word_ham = (self.ham_counts[word] + self.alpha) / (
    ↪self.ham_total_words + self.alpha * v_size)
```

```
                spam_score += math.log(p_word_spam)
                ham_score += math.log(p_word_ham)

        return 1 if spam_score > ham_score else 0

train_text = ["Buy crypto now", "Buy gold now", "Buy milk please"]
labels = [1, 1, 0]

model = NaiveBayesSpamFilter()
model.fit(train_text, labels)

test_email = "crypto Buy"
result = model.predict(test_email)
print(f"The email '{test_email}' is classified as: {'Spam' if result
    ↪== 1 else 'Ham'}")
```

Listing 6: Lower Bound Method (5th Percentile)

Output:

The email 'crypto Buy' is classified as: Spam

8 Week 8: The Art of Evaluation

8.1 Theory and Intuition

You built a classifier in Week 7. But is it any good? The most obvious metric is **Accuracy** ($\frac{\text{Correct Predictions}}{\text{Total Data}}$). However, accuracy is a dangerous liar.

The Accuracy Paradox

Imagine a dataset of 1,000 emails, where only 10 are Spam (1%). If I write a "Dumb Classifier" that simply says **"NOT SPAM"** for every single email, it will be correct 990 times.

- **Accuracy:** 99%.
- **Usefulness:** 0%. (It catches zero spam).

In probability, we call this the **Class Imbalance** problem. To fix this, we need to zoom in on the errors.

8.1.1 The Confusion Matrix

We break down our results into four categories based on the *Truth* vs. the *Prediction*.

	Predicted: Spam	**Predicted: Ham**
Actual: Spam	**True Positive (TP)** (Caught the spam)	**False Negative (FN)** (Missed the spam - Danger!)
Actual: Ham	**False Positive (FP)** (False Alarm)	**True Negative (TN)** (Correctly ignored)

8.1.2 Precision vs. Recall: The Trade-off

These two metrics are like a see-saw. Usually, improving one hurts the other.

- **Precision (Trustworthiness):** $\frac{TP}{TP+FP}$
  "When the model claims an email is spam, how often is it right?"
  High Precision means very few False Alarms (Important if deleting spam automatically).
- **Recall (Sensitivity):** $\frac{TP}{TP+FN}$
  "Of all the spam in the world, what percentage did the model find?"
  High Recall means very few Missed Spams (Important for medical screening).

8.2 Week 8 Deliverables

Week 8 Assignments

Part A: Theoretical Questions

1. **(Easy) The Calculation.** You run your filter on 100 emails.
   - **Reality:** 10 are Spam, 90 are Ham.
   - **Model:** It predicts 8 emails are Spam.
   - **Overlap:** Of the 8 predicted as Spam, 5 were actually Spam.

   Calculate the **Precision**, **Recall**, and **F1-Score**. *(Hint: First find TP, FP, and FN. Note that FN is the spam you missed).*

Section 8: Model Evaluation

Part A: Theoretical Questions

Solution:

Given the dataset parameters ($N = 100$, Actual Spam $= 10$, Predicted Spam $= 8$, $TP = 5$), we derive the Confusion Matrix components:

- **True Positive (TP):** 5
- **False Positive (FP):** $8 - 5 = 3$
- **False Negative (FN):** $10 - 5 = 5$
- **True Negative (TN):** $90 - 3 = 87$

The evaluation metrics are calculated as follows:

$$Precision = \frac{TP}{TP + FP} = \frac{5}{5+3} = 0.625 \text{ (62.5\%)} \tag{12}$$

$$Recall = \frac{TP}{TP + FN} = \frac{5}{5+5} = 0.5 \text{ (50\%)} \tag{13}$$

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \approx 0.555 \text{ (55.5\%)} \tag{14}$$

2. **(Hard) The Bayesian Decision. Context: Cancer Screening.**
   - **False Positive Cost:** Patient anxiety, expensive biopsy ($500).
   - **False Negative Cost:** Late diagnosis, death (Infinite cost).

Solution:

In medical screening, a **False Negative (Type II Error)** is significantly more costly than a False Positive, as it leads to untreated malignancy. To minimize this risk, the **decision threshold should be lowered**. Lowering the threshold prioritizes **Recall** (sensitivity), ensuring that more true cases are captured, even at the expense of lower Precision (increased follow-up tests for healthy individuals).

Your Bayesian model outputs a probability $P(\text{Cancer})$. You must choose a **Threshold** T to make a decision (if $P > T$, predict Cancer). Should you set T high (e.g., 0.9) or low (e.g., 0.1)? Explain your answer using the concepts of Precision and Recall.

Part B: Simulation Task (Coding Metrics from Scratch) *Goal: Do not use Scikit-Learn. Write the math yourself.*
Task:

1. Create two lists representing a test set:
2. **Implement the Confusion Matrix:** Write a function that loops through these lists and counts TP, FP, TN, FN.
3. **Compute Metrics:** Calculate Precision and Recall using your counts.
4. **The "F1-Score":** Sometimes we want a single number that balances both.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Calculate the F1 score for the data above.

5. **Reflection:** Looking at 'y$_{t}rue$`vs`$y_{p}red$`, $thereisaspecificcase(Index3)whereTrue = 1butPred = 0.Whattypeoferroristhis, andwhyisitdangerousinaSpamcontext?$

Part B: Python Implementation

The evaluation metrics, including the confusion matrix counter and the F1-score calculation, have been implemented from scratch to avoid dependency on external libraries.

Source Code: The complete Python implementation for this section is available at the following link:

```python
def get_confusion_matrix(true_y, pred_y):

    tp = fp = tn = fn = 0

    for actual, predicted in zip(true_y, pred_y):
        if actual == 1 and predicted == 1:
            tp += 1
        elif actual == 0 and predicted == 1:
            fp += 1
        elif actual == 0 and predicted == 0:
            tn += 1
        elif actual == 1 and predicted == 0:
            fn += 1

```

```
    return tp, fp, tn, fn


def calculate_metrics(true_y, pred_y):

    tp, fp, tn, fn = get_confusion_matrix(true_y, pred_y)

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0

    recall = tp / (tp + fn) if (tp + fn) > 0 else 0

    if (precision + recall) == 0:
        f1_score = 0
    else:
        f1_score = 2 * (precision * recall) / (precision + recall)

    return {
        "TP": tp, "FP": fp, "TN": tn, "FN": fn,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1_score
    }

if __name__ == "__main__":

    true_labels = [1] * 10 + [0] * 90
    predicted_labels = [1] * 5 + [0] * 5 + [1] * 3 + [0] * 87

    results = calculate_metrics(true_labels, predicted_labels)

    print("--- Model Evaluation Results ---")
    for metric, value in results.items():
        if isinstance(value, float):
            print(f"{metric}: {value:.4f}")
        else:
            print(f"{metric}: {value}")
```

Listing 7: Lower Bound Method (5th Percentile)

Output:

— Model Evaluation Results —
TP: 5
FP: 3
TN: 87
FN: 5
Precision: 0.6250
Recall: 0.5000
F1-Score: 0.5556

9 Week 9: Final Integration and Reflection

9.1 Synthesis

This week is about putting it all together. You have moved from coin flips to spam filters.

9.1.1 The Bayesian Advantage Summary

- **Small Data:** Bayesian methods shine when data is scarce because Priors bridge the gap.
- **Interpretability:** Probabilities mean what we think they mean.
- **Online Learning:** Updating beliefs as new data arrives is natural (Posteriors become Priors).

9.2 Week 9 Deliverables

Final Project Presentation

Prepare a short report (4-5 pages) covering:

1. **Concept Check:** A clear explanation of Priors, Likelihoods, and Posteriors.
2. **The Spam Filter:** Your code, your accuracy/precision metrics, and examples of where it failed.
3. **Reflection:** A paragraph on a real-world scenario (outside of this course) where you would apply Bayesian thinking (e.g., Sports betting, Stock market, Medical diagnosis).